Università degli Studi di Padova

Dipartimento di Matematica "Tullio Levi-Civita"

Corso di Laurea in Informatica

Applicazione *mobile* per la gestione di tour cicloturistici

Tesi di laurea

Relatore

Prof.Davide Bresolin

Laureando

Diego Stafa

Anno Accademico 2021-2022

Diego Stafa: *Applicazione mobile per la gestione di tour cicloturistici,* Tesi di laurea,
© Luglio 2022.

Sommario

Il presente documento descrive il lavoro svolto durante il periodo di stage, effettuato presso l'azienda Moku S.r.l. tra Maggio e Luglio 2022. Lo scopo di tale attività era progettare e sviluppare un'applicazione *mobile* multipiattaforma per la gestione di tour cicloturistici. Gli obiettivi da raggiungere durante il tirocinio erano molteplici.

In primo luogo era richiesto di sviluppare la base dell'applicazione *mobile*, seguendo gli stili e i mockup forniti dal cliente, in secondo luogo veniva richiesto di implementare la funzionalità di condivisione delle posizioni tra i partecipanti di un tour e la visualizzazione in dettaglio di un tour e delle sue caratteristiche, infine era richiesta l'implementazione delle schermate di autenticazione.

Il lavoro si è concentrato principalmente sulle precedenti attività, riuscendo anche a completare schermate opzionali e non richieste. Alla conclusione dello stage, tutte le funzionalità previste per il tirocinio erano concluse.

Il documento è suddiviso in quattro capitoli:

- Il primo capitolo: in questo capitolo viene introdotto il contesto aziendale del tirocinio, fornendo una breve paronamica sull'azienda e la sua storia. Vengono infine introdotti il progetto proposto, le sue caratteristiche e gli obiettivi previsti per tutto lo svolgimento del tirocinio;
- Il secondo capitolo: in questo capitolo vengolo illustrati i processi di sviluppo e gli strumenti adottati per affrontare il progetto;
- Il terzo capitolo: in questo capitolo ci si addentra nello svolgimento del progetto, affrontando le varie fasi percorse, quali l'analisi dei requisiti, la progettazione, la codifica e la validazione del prodotto;
- Il quarto capitolo: in questo capitolo viene attuata una valutazione retrospettiva del tirocinio svolto, andando a stilare gli obiettivi raggiunti e le conoscenze acquisite. Viene infine ritagliato uno spazio per le considerazioni personali sull'esperienza del tirocinio.

Convenzioni tipografiche

- gli acronimi, le abbreviazioni e i termini ambigui o di uso non comune menzionati vengono definiti nel glossario, situato alla fine del presente documento;
- i termini in lingua straniera o facenti parti del gergo tecnico sono evidenziati con il carattere *corsivo*.

Ringraziamenti

Innanzitutto voglio ringraziare con affetto i miei genitori, mia sorella e mio fratello per l'infinito sostegno che mi hanno dato.

Vorrei poi ringraziare il Prof. Davide Bresolin, relatore della mia tesi, per avermi assistito ed essere stato sempre disponibile in questi mesi.

Ringrazio i ragazzi di Moku per l'accoglienza, l'ospitalità, ma soprattutto per le paste, offerti durante il tirocinio.

Infine ringrazio tutti gli amici, i compagni e i colleghi conosciuti in questi anni, in particolar modo Luciano e Samuele.

Padova, Luglio 2022 Diego Stafa

Indice

1 Introduzione **1**
1.1 L'azienda . . . 1
1.1.1 Descrizione . . . 1
1.1.2 Organizzazione interna . . . 1
1.2 Il tirocinio . . . 2
1.2.1 Il progetto proposto . . . 2
1.2.2 Pianificazione iniziale . . . 2
1.2.3 Obiettivi . . . 3

2 Processi e metodologie **5**
2.1 Processi aziendali . . . 5
2.1.1 Metodologia Agile . . . 5
2.2 Strumenti utilizzati . . . 6
2.2.1 Strumenti organizzativi . . . 6
2.2.2 Strumenti di sviluppo . . . 6
2.3 Tecnologie utilizzate . . . 7
2.3.1 Dart . . . 7
2.3.2 Flutter . . . 7
2.3.3 Sqlite . . . 8
2.3.4 JSON . . . 8
2.3.5 Libreria bloc_flutter . . . 8
2.3.6 Libreria google_maps_flutter . . . 8
2.4 Cenni teorici su Flutter e BLoC . . . 9

3 Il progetto: CyclingHero **11**
3.1 Descrizione del progetto . . . 11
3.2 Analisi dei requisiti . . . 11
3.2.1 Casi d'uso . . . 11
3.2.2 Requisiti . . . 29
3.2.3 Tracciamento . . . 33
3.3 Progettazione . . . 37
3.3.1 Architettura di sistema . . . 37
3.4 Codifica . . . 39
3.4.1 Gestione dell'autenticazione . . . 39
3.4.2 Gestione della mappa del gruppo . . . 43
3.4.3 Gestione dell'itinerario . . . 46
3.5 Verifica e validazione . . . 51

4 Conclusione **53**
4.1 Obiettivi raggiunti 53
4.2 Conoscenze acquisite 54
4.3 Considerazioni 54
4.3.1 L'azienda 54
4.3.2 Il cliente 54
4.3.3 Commento finale 54

Acronimi e abbreviazioni **55**

Glossario **57**

Bibliografia **61**

Elenco delle figure

1.1 Il logo di Moku S.r.l. 1

2.1 Rappresentazione del *pattern bloc*. 9

3.1 Gerarchia degli attori. 12

3.2 Casi d'uso relativi all'autenticazione. 13

3.3 Sottocasi d'uso di UC 2. 14

3.4 Sottocasi d'uso di UC 5. 17

3.5 Sottocasi d'uso di UC 6. 19

3.6 Casi d'uso relativi alla gestione della mappa del gruppo. 21

3.7 Sottocasi d'uso di UC 11. 23

3.8 Casi d'uso relativi alla gestione dell'itinerario. 24

3.9 Sottocasi d'uso di UC 14. 25

3.10 Sottocasi d'uso di UC 14.1. 26

3.11 Architettura di CyclingHero. 37

3.12 Da sinistra verso destra: schermata di autenticazione manuale, schermata di autenticazione provvisoria. 40

3.13 Diagramma di sequenza per il processo di autenticazione. 41

3.14 Diagramma di sequenza per il processo di autenticazione provvisoria. . 42

3.15 Schermata della mappa principale del gruppo. 44

3.16 Diagramma di sequenza per il processo di condivisione della posizione. 45

3.17 Da sinistra verso destra: schermata delle giornate di un *tour*, schermata di una giornata in dettaglio, schermata di un *climb* in dettaglio. 47

3.18 Diagramma di sequenza per il processo di calcolo della *route percorsa*. 48

Elenco delle tabelle

1.1 Pianificazione delle attività di inizio tirocinio. 2
1.2 Obiettivi del tirocinio. 3

3.1 Requisiti funzionali. 29
3.2 Requisiti di qualità. 32
3.3 Requisiti di vincolo. 32
3.4 Requisiti prestazionali. 32
3.5 Tracciamento fonte - requisiti. 33
3.6 Tracciamento requisiti - fonte. 34
3.7 Riepilogo dei requisiti. 36
3.8 Soddisfacimento dei requisiti associati alla gestione dell'autenticazione. 42
3.9 Soddisfacimento dei requisiti associati alla gestione della mappa del gruppo. 45
3.10 Soddisfacimento dei requisiti associati alla gestione dell'itinerario. . . . 49

4.1 Completamento degli obiettivi del tirocinio. 53

Capitolo 1

Introduzione

In questo capitolo viene introdotto il contesto aziendale del tirocinio, fornendo una breve paronamica sull'azienda e la sua storia. Vengono infine introdotti il progetto proposto, le sue caratteristiche e gli obiettivi prefissati.

1.1 L'azienda

1.1.1 Descrizione

Figura 1.1: Il logo di Moku S.r.l.

Moku S.r.l. è un'azienda nata in provincia di Treviso nel 2013, sotto l'incubatrice *H-Farm*. Negli anni Moku si è affermata come azienda di consulenza e sviluppo di *software* su commissione, toccando svariati ambiti, tra cui: la biomedicina, la domotica, il turismo e la logistica. L'azienda è fortemente caratterizzata da una metodologia di lavoro Agile e flessibile, a stretto contatto con il cliente, sia questo una piccola *start-up* o una grande impresa.

1.1.2 Organizzazione interna

Dal 2013 Moku ha subito una forte evoluzione interna, se agli inizi la piccola *start-up* si componeva di una manciata di persone, oggi conta invece di un *team* di oltre venti unità, i cui ruoli aziendali si dividono nel seguente modo:

- **CEO**: gestisce le relazioni con i clienti e gli investitori, ricopre la figura amministrativa dell'azienda;

- **CTO**: monitora, valuta e seleziona le tecnologie che possono essere applicate ai prodotti o ai servizi dell'azienda;
- **CFO**: gestisce la pianificazione e le attività finanziarie dell'azienda. Svolge anche il ruolo di archiviazione della documentazione;
- **project manager**: gestisce uno o più progetti, supervisionando e definendo i processi di sviluppo e mantenendo un contatto con il cliente;
- **analista**: si occupa di definire i requisiti di progetto, formalizzando le richieste del cliente attraverso dei requisiti ben definiti;
- **progettista**: prendendo in considerazione il lavoro degli analisti, si occupa di definire l'architettura del sistema in questione, indicandone le componenti e le loro specifiche;
- **sviluppatore**: si occupa dello sviluppo effettivo del prodotto, seguendo le specifiche dettate dai progettisti. All'interno dell'azienda la figura dello sviluppatore viene a sua volta divisa per aree di sviluppo, quali: back-end, front-end e *mobile*;
- **designer**: si occupa di progettare e definire l'interfaccia' dei prodotti.

Durante il tirocinio ho avuto modo di ricoprire la figura di analista, progettista e sviluppatore e collaborare con Alessio Trevisan, il mio *project manager*.

1.2 Il tirocinio

1.2.1 Il progetto proposto

Moku sta sviluppando per conto di una *start-up* il progetto *CyclingHero*: un prototipo di applicazione *mobile* per la gestione di *tour* cicloturistici. Le funzioni chiave dell'applicazione riguardano il monitoraggio di un *tour* e la condivisione delle posizioni dei partecipanti a tale evento. Dato un tour si vuole avere la possibilità di visualizzarne in dettaglio le caratteristiche associate, quindi: le giornate che lo compongono, i percorsi scelti per ogni giornata, lo stato di completamento dei percorsi e gli eventuali climb o POI presenti. Si vuole poi avere la possibilità di condividere la posizione con gli altri partecipanti. All'interno di un *tour* deve essere sempre presente il *CyclingHero*, il pulmino di soccorso che segue il gruppo durante il viaggio. Infine, l'applicazione deve presentare un sistema di autenticazione.

1.2.2 Pianificazione iniziale

Prima dell'inizio del tirocinio si è svolta un'operazione di pianificazione del lavoro insieme all'azienda, la tabella 1.1 riporta l'elenco di attività programmate e la loro durata prevista.

Tabella 1.1: Pianificazione delle attività di inizio tirocinio.

Attività	Settimana	Ore di lavoro previste
Comprensione del sistema e degli obiettivi	1	20
Analisi dei requisiti	1	20

Progettazione	2-3	60
Studio e *setup* dell'ambiente di sviluppo	3	20
Implementazione	4-7	150
Test e validazione	7-8	30
Documentazione	8	20
Totale		320

1.2.3 Obiettivi

Con la tabella 1.2 vengono elencati gli obiettivi stabiliti dall'azienda ospitante prima dell'inizio del tirocinio. Il codice di un obiettivo è nel formato:

`[O][Importanza][Numero]`

dove: `Numero` è un numero incrementale e univoco tra le importanze e `Importanza` definisce l'importanza del requisito secondo i seguenti criteri:

- `O`: se l'obiettivo obbligatorio;
- `D`: se l'obiettivo desiderabile;
- `P`: se l'obiettivo opzionale.

Tabella 1.2: Obiettivi del tirocinio.

Codice	Descrizione
OO1	La gestione e la pianificazione del progetto deve essere gestita tramite una board kanban condivisa.
OO2	Il tirocinante deve praticare un'analisi e una progettazione dell'applicazione *mobile* a partire dai requisiti raccolti.
OO3	Il tirocinante deve sviluppare il modulo di autenticazione dell'applicazione.
OO4	Il tirocinante deve sviluppare il modulo di raccolta e invio dei dati GPS.
OO5	Il tirocinante deve sviluppare il modulo per la gestione dei *tour*, *route* e *POI*.
OD1	Il tirocinante deve essere in grado di coordinarsi con il cliente.
OD2	Il tirocinante deve sviluppare la *suit* di *testing* del prodotto.
OD3	Il tirocinante deve redarre una documentazione completa del prodotto.
OP1	Ulteriori modifiche al prodotto non comprese negli obiettivi obbligatori.

Capitolo 2

Processi e metodologie

In questo capitolo vengolo illustrati i processi di sviluppo e gli strumenti adottati per affrontare il progetto.

2.1 Processi aziendali

2.1.1 Metodologia Agile

Il *team* di Moku lavora utilizzando metodologie *Agile*, basate su *SCRUM*. Il framework *SCRUM* prevede l'impegno di tre regole per la gestione dei processi[1]:

- **trasparenza**: gli aspetti significativi del processo devono essere visibili ai responsabili del lavoro, in un linguaggio formale;

- **ispezione**: i responsabili del lavoro devono poter ispezionare frequentemente, e in modo non invasivo, gli artefatti prodotti, al fine di rilevare anticipatamente eventuali anomalie;

- **adattamento**: la rilevazione di una anomalia deve essere corretta il più velocemente possibile, riducendo al minimo lo spreco di risorse.

Durante il tirocinio ho applicato concretamente i concetti *SCRUM*, lavorando a stretto contatto con il cliente e partecipando a incontri settimanali di sprint review e sprint planning con il *project manager*. Nelle *sprint review* veniva descritto e discusso il lavoro svolto nello *sprint* appena terminato, mentre nei *sprint planning* si andavano a pianificare le attività per lo sprint a venire. Unito a ciò, per il progetto ci siamo serviti delle *board kanban* per formalizzare l'avanzamento dei processi.L'adozione di queste pratiche ci ha permesso di perseguire i principi di trasparenza, ispezione e adattamento *SCRUM*. Infine, Oltre al *framework SCRUM* è stata applicata la pratica del rilascio incrementale di versioni intermedie del prodotto, al fine di ricevere riscontri anche dal cliente.

[1] *Il framework Scrum.* URL: `https://scrumguides.org/scrum-guide.html`.

2.2 Strumenti utilizzati

Di seguito vengono elencati gli strumenti coinvolti nel periodo di tirocinio, descrivendo brevemente il ruolo da loro coperto.

2.2.1 Strumenti organizzativi

Slack

Slack è un'applicazione di messagistica che consente di creare diversi server e canali di comunicazione suddivisi per argomento, mantenendo una separazione intuitiva tra le discussioni. Durante il tirocinio ci siamo avvalsi di due *server* di discussione, il primo per le comunicazioni interne dell'azienda, il secondo per le comunicazioni con il cliente.

Jira

è una *suite* di strumenti *software* di *Issue Tracking System* ormai diventato uno *standard* nel settore informatico, grazie soprattutto alle numerose funzionalità offerte. *Jira* consente di tracciare attività, assegnarle e indicarne la tipologia. Il *software* è inoltre particolarmente adatto per l'azienda e la sua metodologia di sviluppo, fornendo *board* preconfigurate per l'attuazione di strategie *agili* - *SCRUM*.

Trello

Trello è uno strumento *web* basato sul modello *kanban* per la gestione di progetti. Durante il tirocinio *Trello* è stato sfruttato sia per tracciare le attività stabilite, sia per fornire al cliente un riscontro visivo sullo stato del progetto.

Google Meet

Google Meet è un semplice servizio di comunicazione video. Durante il tirocinio *Google Meet* è stato sfruttato sia per gli incontri periodici con il cliente, sia per le comunicazioni interne del *team* nel caso in cui non ci fosse la disponibilità fisica.

2.2.2 Strumenti di sviluppo

Git

è un *software* di controllo di versione distribuito, volto ad agevolare il coordinamento tra più persone che lavorano su un stesso insieme di file. Durante il tirocinio *git* è stato sfruttato per il versionamento dei sorgenti in locale.

BitBucket

Bitbucket è un servizio *cloud* di *hosting* per progetti basati sui *VCS mercurial* o *git*. Essendo *Moku* un'azienda che sfrutta estensivamente il servizio da molti anni, per il progetto è stato utilizzato *Bitbucket*.

Github

Github è un servizio *cloud* di *hosting* per progetti basati su *git*. Durante il progetto è stato utilizzato per sincronizzare i sorgenti con il *repository* il cliente.

VScode

VScode è un editor di testo estremamente estensibile che offre molte utili funzioni, tra cui: *debugging*, *syntax highlighting* e autocompletamento. Per il progetto è stato scelto *VScode* in quanto presenta delle utili estensioni per lo sviluppo in Flutter.

2.3 Tecnologie utilizzate

Lo scopo di questa sezione è elencare e descrivere le tecnologie utilizzate nello sviluppo del progetto, illustrando anche i loro vantaggi e svantaggi.

2.3.1 Dart

Dart è un linguaggio fortemente tipizzato basato sul paradigma a oggetti e sviluppato da Google nel 2011. Il linguaggio prende forte ispirazione da altri famosi linguaggi, come *Java* e *Javascript*, ma riesce a distinguersi grazie alle molte comodità che il linguaggio offre.
Vantaggi:

- ha una sintassi molto semplice;
- il gestore dei pacchetti, *pub*;
- la sound-null safety;
- la possibilità di *transcompilare* su diverse piattaforme;
- una *community* molto attiva.

Svantaggi:

- rispetto ad altri linguaggi ormai affermati, *Dart* presenta un ristretto ecosistema di librerie.

L'utilizzo di *Dart* è stato rispettato come espresso nei vincoli.

2.3.2 Flutter

Flutter è un *framework* grafico per la creazione di interfacce grafiche multipiattaforma, rilasciato da *Google* nel 2017.
Vantaggi:

- permette di sviluppare applicazioni multipiattaforma;
- hot-reloading.

Svantaggi:

- le applicazioni *Flutter* hanno prestazioni generalmente peggiori rispetto alle controparti native.

L'utilizzo di *Flutter* è stato rispettato come espresso nei vincoli.

2.3.3 Sqlite

Sqlite è un database relazionale compatibile con SQL. A differenza di altri sistemi basati su SQL come *MySQL* e *PostgreSQL*, *SQLite* non utilizza un'architettura *clientserverg*. L'intero programma è contenuto in una libreria, che è integrata nelle applicazioni[2].
Vantaggi:

- offre prestazioni e tempi di risposta molto buone;
- può interpretare campi JSON;
- è semplice.

Svantaggi:

- non supporta le viste;
- non supporta la concorrenza.

2.3.4 JSON

JSON è il formato dati scelto per lo scambio di dati fra l'applicazione e il *back-end*. L'utilizzo di *JSON* è stato rispettato come espresso nei vincoli.

2.3.5 Libreria bloc_flutter

bloc_flutter è una libreria che permette di implementare facilmente il *pattern* Business LOgic Component (BLOC) in applicazioni *flutter*. **Vantaggi**:

- offre un'elevata robustezza e scalabilità;
- nasconde i dettagli implementativi del *pattern* fornendo un'API semplice.

2.3.6 Libreria google_maps_flutter

Per la visualizzazione delle mappe per il progetto è stato deciso l'utilizzo della libreria *google_maps_flutter*. Altre librerie, come *flutter_map* o *mapbox*, sono state scartate dopo un'attenta analisi.
Vantaggi:

- la documentazione e la *community* sono molto ampie;
- permette di implementare il raggruppamento dinamico di più punti nella mappa[3].

[2] *Documentazione ufficiale di Sqlite*. URL: `https://www.sqlite.org/docs.html`.
[3] *Clustering dei dati con google$_{m}aps_{f}lutter$*. URL: `https://pub.dev/packages/google_maps_cluster_manager`.

2.4 Cenni teorici su Flutter e BLoC

Si può definire un'applicazione *Flutter* come un'insieme di schermate. Una schermata è un elemento formato da uno o più *widget*, i quali rappresentano gli elementi visivi e grafici che vanno a comporre l'applicazione[4], i *widget* sono rappresentati nel *framework* come delle semplici classi. Un widget può essere principalmente di due tipi:

- **Stateless**: nel caso in cui non mantenga uno stato interno;
- **Stateful**: nel caso in cui mantenga uno stato interno.

Flutter non impone una determinata architettura per la costruzione delle schermate, ma affida allo sviluppatore il compito di ciò, andando però a suggerire diversi *pattern* e tra questi il più famoso è il *BLoC*. In *Flutter* il *BLoC pattern* si basa sul concetto di *business logic component*, cioè un elemento che emette stati e riceve eventi di una schermata[5]. In generale la funzione di un *bloc* è quella di gestire lo stato e gli eventi di un elemento, separando quindi il lato strutturale di una schermata da quello comportamentale. La figura 2.1 illustra il normale processo di comunicazione per le applicazioni *flutter* basate sul *pattern bloc*.

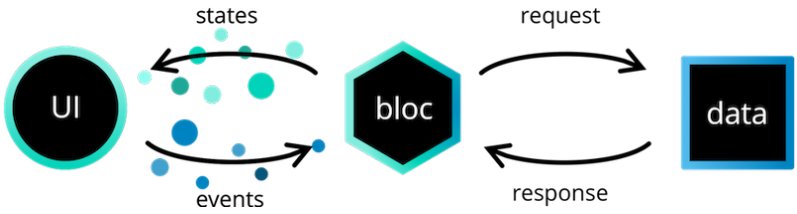

Figura 2.1: Rappresentazione del *pattern bloc*.

[4] *Documentazione ufficiale di Flutter*. URL: `https://docs.flutter.dev/`.
[5] *Il pattern BLoC*. URL: `https://bloclibrary.dev/#/gettingstarted`.

Capitolo 3

Il progetto: CyclingHero

In questo capitolo ci si addentra nello svolgimento del progetto, affrontando le varie fasi percorse, quali l'analisi dei requisiti, la progettazione, la codifica e la validazione del prodotto.

3.1 Descrizione del progetto

Il progetto *CyclingHero* pone l'obiettivo di sviluppare un applicazione multipiattaforma per gestire, pianificare e monitorare *tour* cicloturistici.
Dopo una prima fase di autenticazione, l'utente deve poter accedere alla mappa del *tour*, dove può sincronizzarsi con le posizioni del gruppo, decidendo di inviare periodicamente la sua posizione geografica, sempre in questa schermata l'utente deve poter visualizzare le distanze tra gli altri utenti. In aggiunta alla condivisione delle posizioni, l'utente deve poter monitorare l'itinerario, visualizzandone le sue informazioni, quindi le giornate di cui è composto, le *route* di ogni giornata ed eventuali *climb* e *POI*. Infine deve essere presente un meccanismo per il calcolo della *percorsa*,

3.2 Analisi dei requisiti

3.2.1 Casi d'uso

Introduzione

In questa sezione sono descritti i principali casi d'uso dell'applicazione, un caso d'uso è metodo formale di rappresentazione dei requisiti orientato alla figura dell'attore, cioè colui che interagisce con l'applicazione. Ogni caso d'uso successivamente descritto presenta la seguente struttura:

- **diagramma UML** (opzionale): per rappresentare le relazioni con altri casi d'uso;
- **intestazione**, nel formato: `UC [codice] - [titolo]`;
- **attore/i primario/i**: entità esterna al sistema e che vi interagisce;
- **attore/i secondario/i** (opzionale): entità esterna al sistema di supporto per il raggiungimento dell'obiettivo dell'attore primario;

- **descrizione**: breve descrizione del caso d'uso;
- **precondizioni**: le condizioni del sistema prima del verificarsi del caso d'uso;
- **scenario principale**: elenco puntato che scandisce il flusso degli eventi;
- **postcondizioni**: le condizioni del sistema dopo che si è verificato il caso d'uso;
- **estensioni** (opzionali): gli scenari alternativi, che iniziano la loro esecuzione interrompendo il caso d'uso che ne ha verificato le precondizioni.

Il codice di un caso d'uso senza padre è un numero progressivo univoco tra i casi d'uso. Il codice di un sottocaso d'uso è nel formato:

`[codice_padre].[numero_figlio]`

dove:

- `codice_padre` è il codice che identifica in maniera univoca il padre;
- `numero_figlio` è un numero progressivo univoco tra i figli dello stesso padre.

Attori

Attori primari L'attore primario è la figura principale che si interfaccia all'applicazione, come illustrato nella figura 3.1, sono stati individuati due attori per l'applicazione:

- **Utente non autenticato**: indica un utente che non ha ancora effettuato l'autenticazione, interagisce solo con il sistema di autenticazione dell'applicazione;
- **Utente autenticato**: indica un utente che ha effettuato l'autenticazione, interagisce con l'applicazione, condividendo la sua posizione e gestendo l'itinerario.

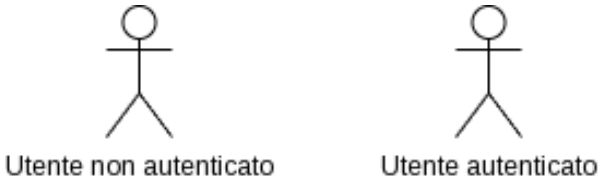

Figura 3.1: Gerarchia degli attori.

Elenco dei casi d'uso

Segue l'elenco dei casi d'uso ritrovati, l'elenco è diviso essenzialmente in tre aree:

- casi d'uso per l'autenticazione (figura 3.2);
- casi d'uso per la mappa dell'itinerario (figura 3.6);
- casi d'uso per la visualizzazione in dettaglio dell'itinerario (figura 3.8).

I casi d'uso di autenticazione si rivolgono alle azioni di autenicazione, di registrazione e di recupero password, i casi d'uso per la mappa dell'itinerario descrivono le azioni di condivisione della posizione e visualizzazione della mappa. Infine i casi d'uso per la visualizzazione in dettaglio dell'itinerario specificano le azioni di visualizzazione sulle componenti di un itinerario, quali: le giornate, le *route*, i *climb* e altre informazioni minori.

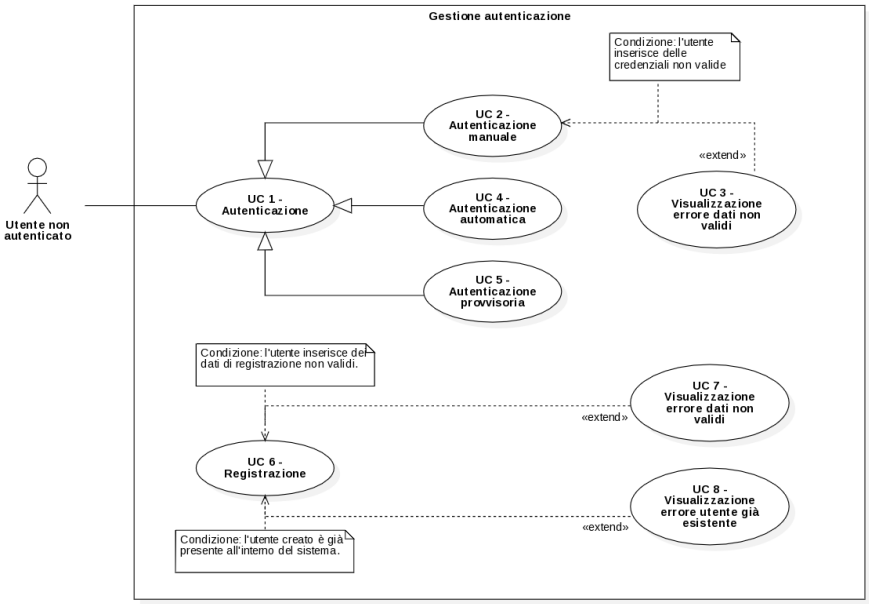

Figura 3.2: Casi d'uso relativi all'autenticazione.

UC 1 - Autenticazione

- **Attore primario**: utente non autenticato;
- **Descrizione**: l'utente deve potersi autenticare e accedere al proprio account;
- **Scenario principale**:
  1. l'utente apre l'applicazione;
  2. l'utente si autentica normalmente (**UC 1.1**), in modo automatico (**UC 1.3**), se aveva precedentemente memorizzato la sessione, o accede con l'autenticazione provvisoria (**UC 2**).
- **Precondizioni**: l'utente non è ancora autenticato nel sistema;
- **Postcondizioni**: l'utente si è autenticato nel sistema.

UC 1.1 - Autenticazione manuale

- **Attore**: utente non autenticato;
- **Descrizione**: l'utente deve potersi autenticare e accedere manualmente al proprio account;
- **Scenario principale**:
  1. l'utente apre l'applicazione;
  2. l'utente inserisce la propria *e-mail* (**UC 1.1.1**);
  3. l'utente inserisce la propria password (**UC 1.1.2**);

4. se l'utente vuole, seleziona la casella di memorizzazione della sessione (**UC 1.4**);
5. l'utente conferma i dati inseriti (**UC 1.2**).

- **Precondizioni**: l'utente non è ancora autenticato nel sistema;
- **Postcondizioni**:
  - l'utente si è autenticato nel sistema;
  - se l'utente ha selezionato la casella di memorizzazione della sessione, al prossimo accesso al sito verrà autenticato automaticamente.
- **Estensioni**: l'utente inserisce delle credenziali non valide e viene mostrato un messaggio d'errore (**UC 3**).

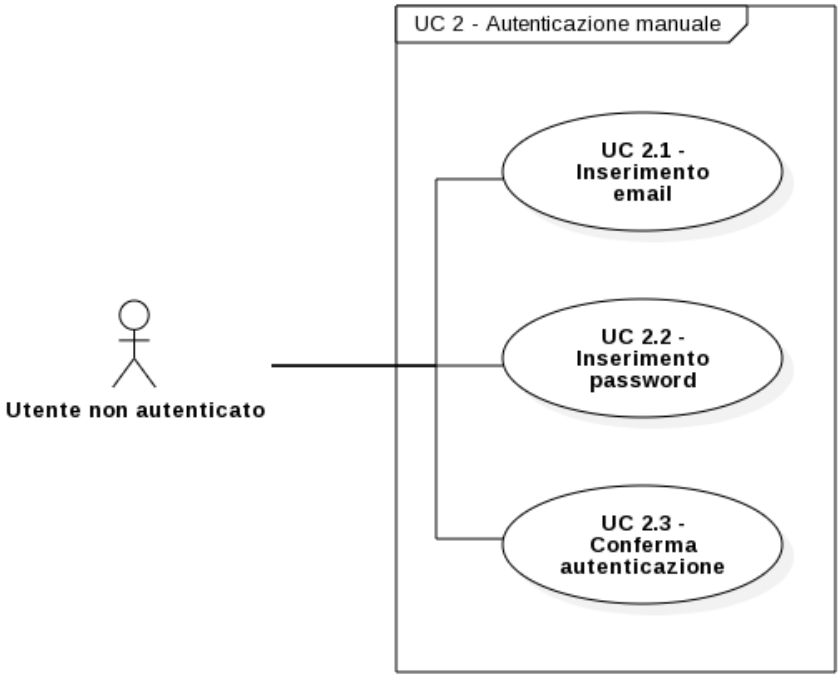

Figura 3.3: Sottocasi d'uso di UC 2.

UC 1.1.1 - Inserimento email

- **Attore primario**: utente non autenticato;
- **Descrizione**: l'utente deve poter inserire la propria *e-mail* durante l'attività di autenticazione manuale;
- **Scenario principale**:
  1. l'utente seleziona il campo relativo al nome utente;

2. l'utente digita il proprio nome utente.

- **Precondizioni**: l'utente sta svolgendo l'attività di autenticazione;
- **Postcondizioni**: l'utente ha compilato il campo relativo al nome utente.

UC 1.1.2 - Inserimento password

- **Attore primario**: utente non autenticato;
- **Descrizione**: l'utente deve poter inserire la propria password durante l'attività di autenticazione;
- **Scenario principale**:
  1. l'utente seleziona il campo relativo alla password;
  2. l'utente digita la propria password.
- **Precondizioni**: l'utente sta svolgendo l'attività di autenticazione;
- **Postcondizioni**: l'utente ha compilato il campo relativo alla password.

UC 1.2 - Conferma autenticazione

- **Attore primario**: utente non autenticato;
- **Descrizione**: l'utente deve poter confermare i dati inseriti e tentare di autenticarsi;
- **Scenario principale**: l'utente conferma l'azione di autenticazione;
- **Precondizioni**: l'utente sta svolgendo l'attività di autenticazione;
- **Postcondizioni**: l'utente ha tentato di autenticarsi utilizzando i dati inseriti.

UC 1.3 - Autenticazione automatica

- **Attore primario**: utente non autenticato;
- **Descrizione**: l'utente deve poter essere autenticato automaticamente se ha memorizzato una sessione;
- **Scenario principale**:
  1. l'utente accede al sito;
  2. l'utente viene automaticamente autenticato.
- **Precondizioni**: l'utente ha una sessione valida memorizzata;
- **Postcondizioni**: l'utente è autenticato.

UC 1.4 - Memorizzazione sessione

- **Attore primario**: utente non autenticato;
- **Descrizione**: l'utente deve poter richiedere la memorizzazione della sessione per effettuare automaticamente il autenticazione al prossimo accesso al sito;
- **Scenario principale**: l'utente seleziona la casella di memorizzazione della sessione;
- **Precondizioni**: l'utente sta svolgendo l'attività di autenticazione;
- **Postcondizioni**: l'utente ha richiesto la memorizzazione della sessione.

UC 2 - Autenticazione provvisoria

- **Attore primario**: utente non autenticato;
- **Descrizione**: l'utente deve potersi autenticare e accedere a un account preesistente;
- **Scenario principale**:
  1. l'utente apre l'applicazione;
  2. l'utente seleziona un utente preesistente (**UC 2.1**);
  3. l'utente si autentica con l'utente selezionato (**UC 2.2**).
- **Precondizioni**: l'utente non è ancora autenticato nel sistema;
- **Postcondizioni**: l'utente si è autenticato nel sistema.

UC 2.1 - Selezione utente

- **Attore primario**: utente non autenticato;
- **Descrizione**: l'utente deve poter selezionare un utente preesistente nel sistema;
- **Scenario principale**: l'utente seleziona un utente preesistente nel sistema;
- **Precondizioni**: l'utente sta svolgendo l'attività di autenticazione provvisoria;
- **Postcondizioni**: l'utente ha compilato il campo relativo alla password.

UC 2.2 - Conferma autenticazione

- **Attore primario**: utente non autenticato;
- **Descrizione**: l'utente deve poter confermare i dati inseriti e tentare di autenticarsi;
- **Scenario principale**: l'utente conferma l'azione di autenticazione;
- **Precondizioni**: l'utente sta svolgendo l'attività di autenticazione;
- **Postcondizioni**: l'utente ha tentato di autenticarsi utilizzando i dati inseriti.

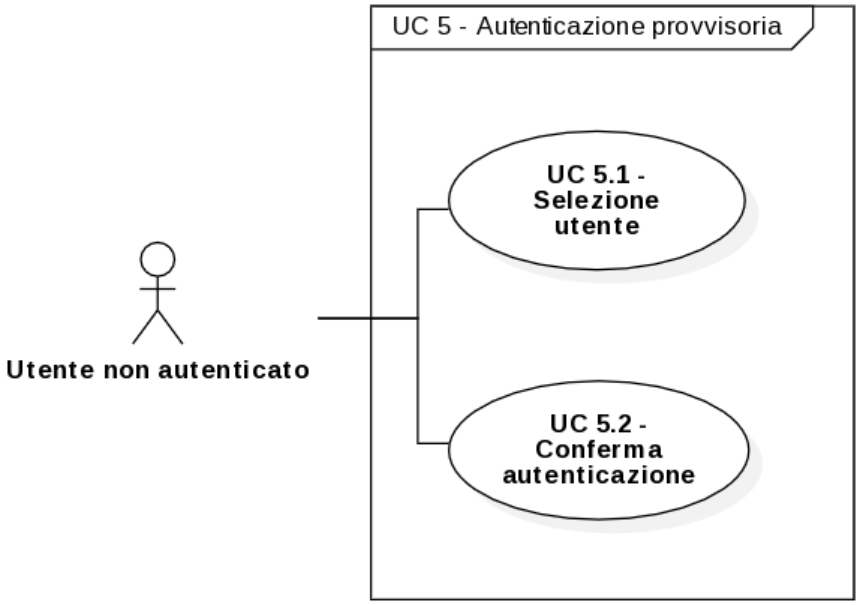

Figura 3.4: Sottocasi d'uso di UC 5.

UC 3 - Visualizzazione errore dati inseriti non validi

- **Attore primario**: utente non autenticato;
- **Descrizione**: l'utente deve ricevere un errore a seguito dell'inserimento di credenziali non valide durante l'attività di autenticazione manuale;
- **Scenario principale**: l'utente visualizza un messaggio di errore;
- **Precondizioni**: l'utente sta svolgendo l'attività di autenticazione manuale e inserisce delle credenziali non valide;
- **Postcondizioni**: l'utente ha ricevuto un messaggio d'errore.

UC 4 - Registrazione

- **Attore primario**: utente non autenticato;
- **Descrizione**: l'utente deve potersi registrare all'interno del sistema;
- **Precondizioni**: l'utente non è ancora autenticato nel sistema;
- **Scenario principale**:
  1. l'utente apre l'applicazione;
  2. l'utente clicca il pulsante di registrazione (**UC 4.1**);
  3. l'utente inserisce il suo nome (**UC 4.2**);
  4. l'utente inserisce il suo cognome (**UC 4.3**);

5. l'utente inserisce la sua password (**UC 4.4**);
6. l'utente inserisce nuovamente la password inserita in precedenza (**UC 4.5**);
7. l'utente conferma i dati inseriti (**UC 4.6**).

- **Postcondizioni**: l'utente è registrato e autenticato nel sistema;
- **Estensioni**:
  1. l'utente inserisce dei dati non validi e viene mostrato un messaggio d'errore (**UC 5**);
  2. l'utente creato è già presente all'interno del sistema e viene mostrato un messaggio d'errore (**UC 6**).

UC 4.1 - Inserimento nome

- **Attore primario**: utente non autenticato;
- **Descrizione**: l'utente deve poter inserire il proprio nome durante l'attività di registrazione;
- **Precondizioni**: l'utente sta svolgendo l'attività di registrazione;
- **Scenario principale**:
  1. l'utente seleziona il campo relativo al nome;
  2. l'utente digita il proprio nome.
- **Postcondizioni**: l'utente ha compilato il campo relativo al nome.

UC 4.2 - Inserimento cognome

- **Attore primario**: utente non autenticato;
- **Descrizione**: l'utente deve poter inserire il proprio cognome durante l'attività di registrazione;
- **Scenario principale**:
  1. l'utente seleziona il campo relativo al cognome;
  2. l'utente digita il proprio cognome.
- **Precondizioni**: l'utente sta svolgendo l'attività di registrazione;
- **Postcondizioni**: l'utente ha compilato il campo relativo al cognome.

UC 4.3 - Inserimento email

- **Attore primario**: utente non autenticato;
- **Descrizione**: l'utente deve poter inserire la propria *e-mail* durante l'attività di registrazione;
- **Precondizioni**: l'utente sta svolgendo l'attività di registrazione;

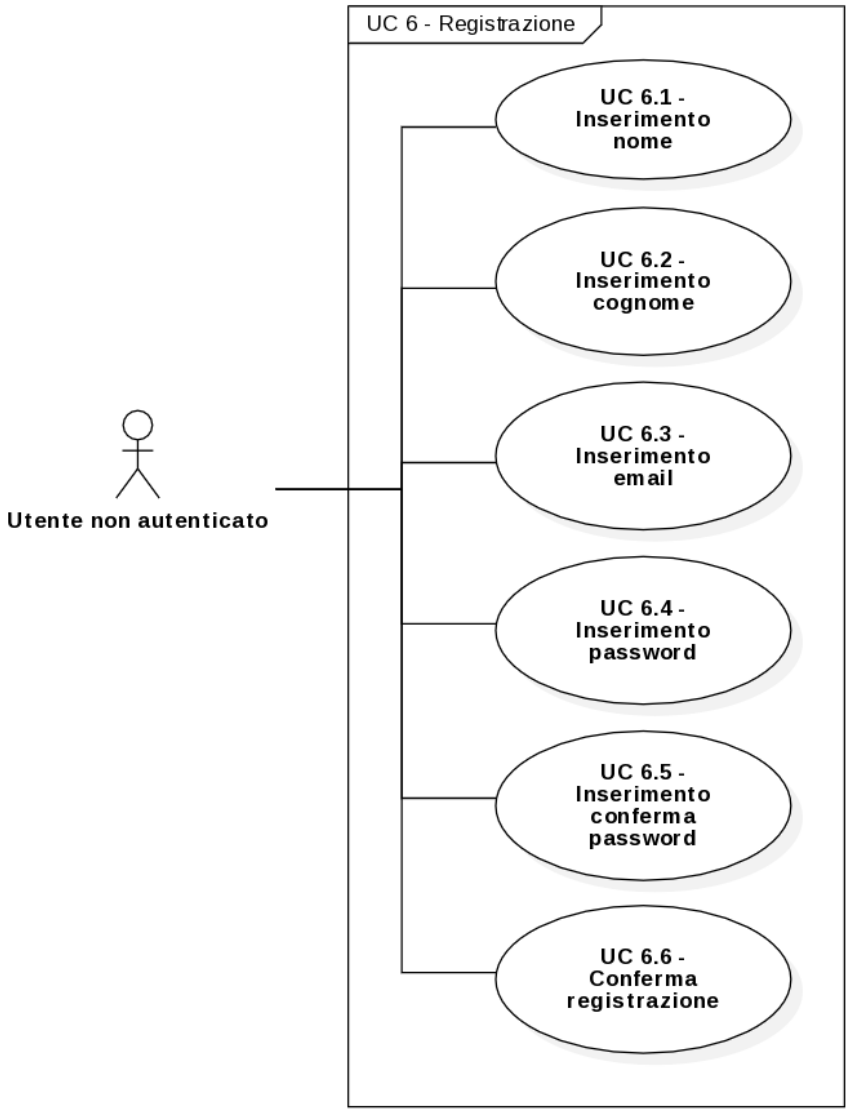

Figura 3.5: Sottocasi d'uso di UC 6.

- **Scenario principale**:
  1. l'utente seleziona il campo relativo all'*email*;
  2. l'utente digita la propria *email*.
- **Postcondizioni**: l'utente ha compilato il campo relativo alla *email*.

UC 4.4 - Inserimento password

- **Attore primario**: utente non autenticato;

- **Descrizione**: l'utente deve poter inserire la propria password durante l'attività di autenticazione;
- **Precondizioni**: l'utente sta svolgendo l'attività di autenticazione;
- **Scenario principale**:
  1. l'utente seleziona il campo relativo alla password;
  2. l'utente digita la propria password.
- **Postcondizioni**: l'utente ha compilato il campo relativo alla password.

UC 4.5 - Inserimento conferma password

- **Attore primario**: utente non autenticato;
- **Descrizione**: l'utente deve poter inserire nuovamente la propria password durante l'attività di registrazione;
- **Precondizioni**: l'utente sta svolgendo l'attività di registrazione;
- **Scenario principale**:
  1. l'utente seleziona il campo relativo alla conferma della password;
  2. l'utente digita nuovamente la propria password.
- **Postcondizioni**: l'utente ha compilato il campo relativo alla conferma della password.

UC 4.6 - Conferma registrazione

- **Attore primario**: utente non autenticato;
- **Descrizione**: l'utente conferma i dati inseriti relativi alla registrazione;
- **Precondizioni**: l'utente sta svolgendo l'attività di registrazione;
- **Scenario principale**: l'utente conferma i dati inseriti;
- **Postcondizioni**: l'utente viene creato nel sistema.

UC 5 - Visualizzazione errore dati inseriti non validi

- **Attore primario**: utente non autenticato;
- **Descrizione**: l'utente deve ricevere un errore a seguito dell'inserimento di credenziali non valide durante l'attività di registrazione;
- **Precondizioni**: l'utente sta svolgendo l'attività di registrazione e inserisce delle credenziali non valide;
- **Scenario principale**: l'utente visualizza un messaggio di errore;
- **Postcondizioni**: l'utente ha ricevuto un messaggio d'errore.

UC 6 - Visualizzazione errore utente già esistente

- **Attore primario**: utente non autenticato;
- **Descrizione**: l'utente deve ricevere un errore per aver tentato di creare un utente già presente nel sistema;
- **Precondizioni**: l'utente sta svolgendo l'attività di registrazione;
- **Scenario principale**: l'utente tenta di aggiungere un utente già presente nel sistema;
- **Postcondizioni**: l'utente ha ricevuto un messaggio d'errore.

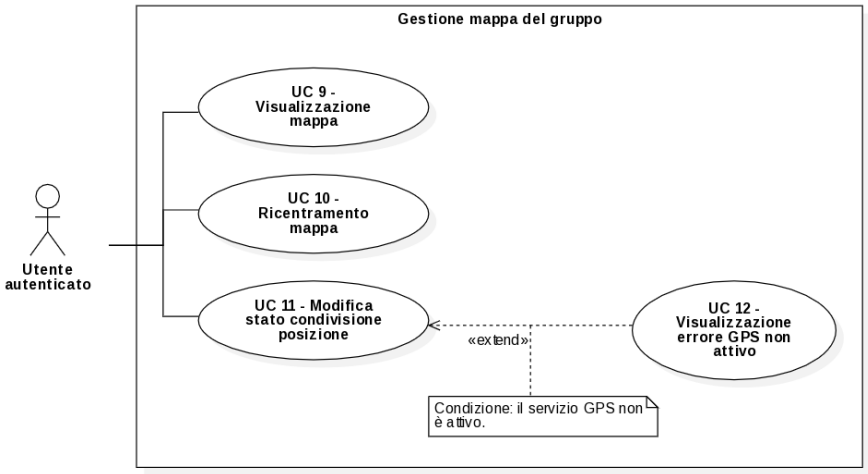

Figura 3.6: Casi d'uso relativi alla gestione della mappa del gruppo.

UC 7 - Visualizzazione mappa

- **Attore primario**: utente autenticato;
- **Descrizione**: l'utente deve poter visualizzare tutti i partecipanti e il loro nome nella mappa, evidenziando l'utente stesso e il pulmino di soccorso;
- **Precondizioni**: l'utente si trova nella schermata di condivisione della posizione;
- **Scenario principale**:
  1. l'utente visualizza tutti i partecipanti nella mappa, con il loro nome, distinguendo sè stesso e il pulmino di soccorso (**UC 7.1**);
  2. l'utente clicca su un partecipante e visualizza la distanza da esso (**UC 7.2**).
- **Postcondizioni**: l'utente visualizza la mappa del gruppo.

UC 7.1 - Visualizzazione partecipanti

- **Attore primario**: utente autenticato;
- **Descrizione**: l'utente deve poter visualizzare i partecipanti nella mappa;
- **Precondizioni**: l'utente sta svolgendo l'attività di visualizzazione della mappa;
- **Scenario principale**: l'utente visualizza i partecipanti dell'itinerario nella mappa;
- **Postcondizioni**: l'utente sta svolgendo l'attività di visualizzazione della mappa.

UC 7.2 - Visualizzazione distanze

- **Attore primario**: utente autenticato;
- **Descrizione**: l'utente deve poter visualizzare la distanza tra lui e un altro partecipante;
- **Precondizioni**: l'utente sta svolgendo l'attività di visualizzazione della mappa;
- **Scenario principale**: l'utente clicca su un partecipante nella mappa;
- **Postcondizioni**: l'utente visualizza la distanza tra lui e il partecipante indicato.

UC 8 - Ricentramento mappa

- **Attore primario**: utente autenticato;
- **Descrizione**: l'utente deve poter ricentrare la mappa nella posizione in cui si trova;
- **Precondizioni**: l'utente non è centrato nella mappa;
- **Scenario principale**: l'utente clicca il pulsante di ricentramento della posizione;
- **Postcondizioni**: l'utente è centrato nella mappa.

UC 9 - Modifica stato condivisione posizione

- **Attore primario**: utente autenticato;
- **Descrizione**: l'utente deve poter condividere la sua posizione con il resto del gruppo;
- **Precondizioni**: l'utente si trova nella schermata di condivisione della posizione;
- **Scenario principale**: l'utente attiva (**UC 9.1**) o disattiva (**UC 9.2**) la condivisione della posizione;
- **Postcondizioni**: l'utente condivide la sua posizione con il resto del gruppo;
- **Estensioni**: l'utente non ha il servizio GPS attivo e viene mostrato un messaggio d'errore (**UC 10**).

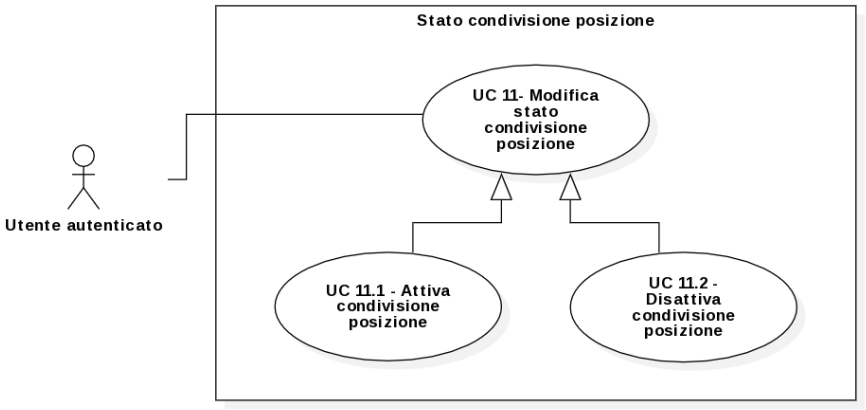

Figura 3.7: Sottocasi d'uso di UC 11.

UC 9.1 - Attiva condivisione posizione

- **Attore primario**: utente autenticato;
- **Descrizione**: l'utente deve poter attivare la condivisione della posizione;
- **Precondizioni**:
  1. l'utente sta svolgendo l'attività di modifica dello stato di condivisione posizione;
  2. la condivisione della posizione è disattivata.
- **Scenario principale**: l'utente attiva la condivisione della posizione;
- **Postcondizioni**: la condivisione della posizione è attivata.

UC 9.2 - Disattiva condivisione posizione

- **Attore primario**: utente autenticato;
- **Descrizione**: l'utente deve poter disattivare la condivisione della posizione;
- **Precondizioni**:
  1. l'utente sta svolgendo l'attività di modifica dello stato di condivisione posizione;
  2. la condivisione della posizione è attivata.
- **Scenario principale**: l'utente disattiva la condivisione della posizione;
- **Postcondizioni**: la condivisione della posizione è disattivata.

UC 10 - Visualizzazione errore GPS non attivo

- **Attore primario**: utente autenticato;
- **Descrizione**: l'utente deve ricevere un errore a seguito dell'attivazione della condivisione della posizione nel caso in cui il GPS non sia attivato;
- **Precondizioni**: l'utente sta svolgendo l'attività di modifica stato condivisione posizione;
- **Scenario principale**: l'utente visualizza un messaggio di errore;
- **Postcondizioni**: l'utente ha ricevuto un messaggio d'errore.

UC 11 - Visualizzazione giornate itinerario

- **Attore primario**: utente autenticato;
- **Descrizione**: l'utente deve poter visualizzare l'itinerario corrente;
- **Precondizioni**: l'utente si trova nella schermata degli itinerari;
- **Scenario principale**: l'utente visualizza tutte le giornate dell'itinerario corrente;
- **Postcondizioni**: l'utente visualizza l'itinerario corrente.

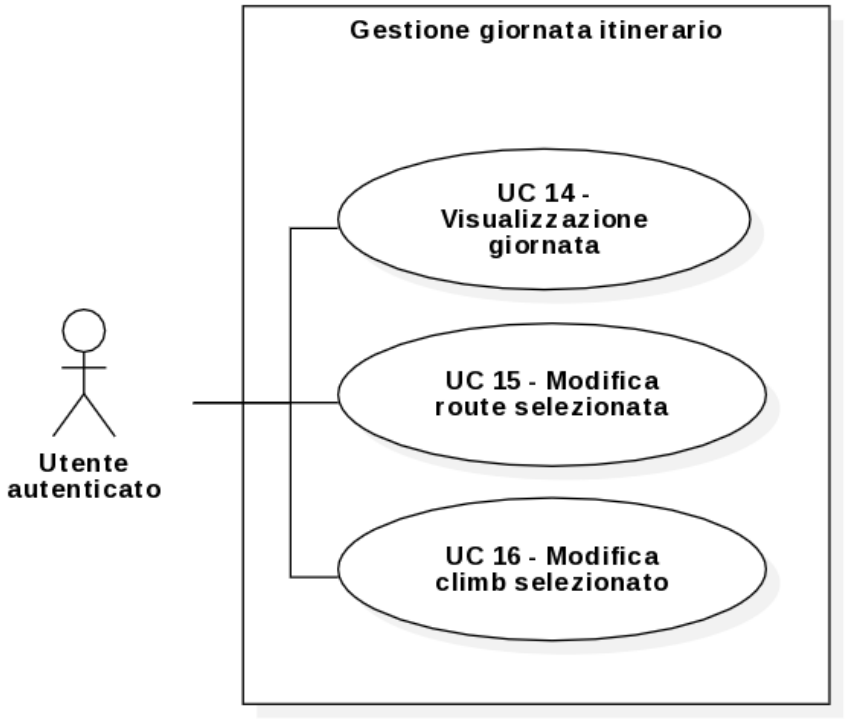

Figura 3.8: Casi d'uso relativi alla gestione dell'itinerario.

UC 12 - Visualizzazione giornata

- **Attore primario**: utente autenticato;
- **Descrizione**: l'utente deve poter visualizzare una giornata di un itinerario in dettaglio;
- **Precondizioni**: l'utente si trova nella schermata degli itinerari;
- **Scenario principale**:
  1. l'utente visualizza la mappa della giornata (**UC 12.1**);
  2. l'utente visualizza un *POI* in dettaglio (**UC 12.2**);
  3. l'utente visualizza un *climb* in dettaglio (**UC 12.3**).
- **Postcondizioni**: l'utente visualizza una giornata in dettaglio.

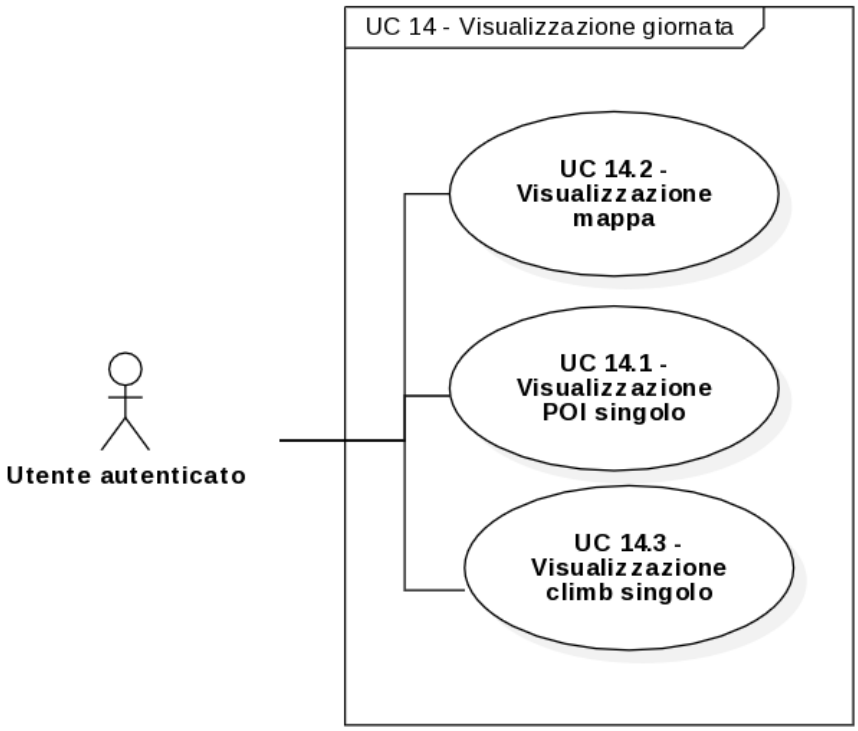

Figura 3.9: Sottocasi d'uso di UC 14.

UC 12.1 - Visualizzazione mappa

- **Attore primario**: utente autenticato;
- **Descrizione**: l'utente deve poter visualizzare la mappa della giornata;
- **Precondizioni**: l'utente si trova nella schermata degli itinerari;

- **Scenario principale**:
  1. l'utente visualizza i partecipanti (**UC 12.1.1**);
  2. l'utente visualizza tutti i POI (**UC 12.1.2**);
  3. l'utente visualizza il *climb* al momento selezionato (**UC 12.1.3**);
  4. l'utente visualizza la *route* al momento selezionata (**UC 12.1.4**);
  5. l'utente visualizza la route percorsa dai partecipanti (**UC 12.1.5**).
- **Postcondizioni**: l'utente visualizza la mappa di una giornata.

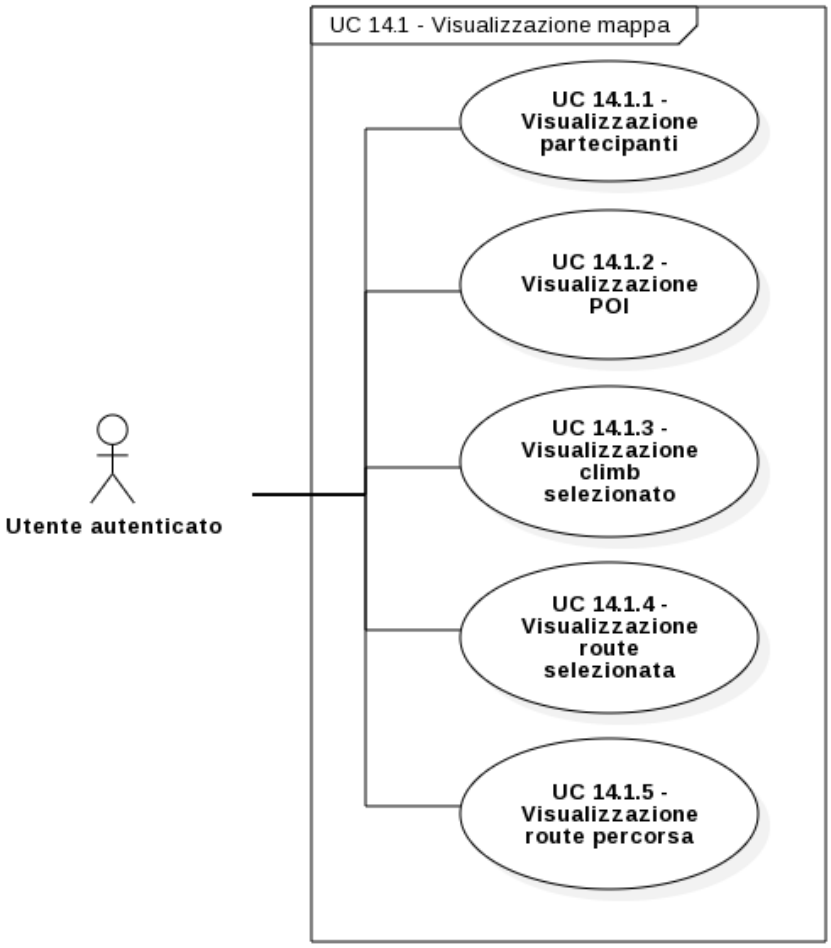

Figura 3.10: Sottocasi d'uso di UC 14.1.

UC 12.1.1 - Visualizzazione partecipanti

- **Attore primario**: utente autenticato;

- **Descrizione**: l'utente deve poter visualizzare i partecipanti nella mappa;
- **Precondizioni**: l'utente sta svolgendo l'attività di visualizzazione della mappa;
- **Scenario principale**: l'utente visualizza i partecipanti dell'itinerario nella mappa;
- **Postcondizioni**: l'utente sta svolgendo l'attività di visualizzazione della mappa.

UC 12.1.2 - Visualizzazione POI

- **Attore primario**: utente autenticato;
- **Descrizione**: l'utente deve poter visualizzare i *POI* nella mappa;
- **Precondizioni**: l'utente sta svolgendo l'attività di visualizzazione della mappa;
- **Scenario principale**: l'utente visualizza tutti i *POI* dell'itinerario nella mappa;
- **Postcondizioni**: l'utente sta svolgendo l'attività di visualizzazione della mappa.

UC 12.1.3 - Visualizzazione *climb* selezionato

- **Attore primario**: utente autenticato;
- **Descrizione**: l'utente deve poter visualizzare il *climb* selezionato nella mappa;
- **Precondizioni**: l'utente sta svolgendo l'attività di visualizzazione della mappa;
- **Scenario principale**: l'utente visualizza il *climb* selezionato nella mappa;
- **Postcondizioni**: l'utente sta svolgendo l'attività di visualizzazione della mappa.

UC 12.1.4 - Visualizzazione *route* selezionata

- **Attore primario**: utente autenticato;
- **Descrizione**: l'utente deve poter visualizzare la *route* selezionata nella mappa;
- **Precondizioni**: l'utente sta svolgendo l'attività di visualizzazione della mappa;
- **Scenario principale**: l'utente visualizza la *route* selezionata nella mappa;
- **Postcondizioni**: l'utente sta svolgendo l'attività di visualizzazione della mappa.

UC 12.1.5 - Visualizzazione *route* percorsa

- **Attore primario**: utente autenticato;
- **Descrizione**: l'utente deve poter visualizzare la *route* percorsa da un partecipante;
- **Precondizioni**: l'utente sta svolgendo l'attività di visualizzazione mappa;
- **Scenario principale**: l'utente clicca su un partecipante nella mappa;
- **Postcondizioni**: l'utente visualizza la *route* percorsa.

UC 12.2 - Visualizzazione POI singolo

- **Attore primario**: utente autenticato;
- **Descrizione**: l'utente deve poter visualizzare un *POI* in dettaglio;
- **Precondizioni**: l'utente si trova nella schermata delle giornate;
- **Scenario principale**:
  1. l'utente clicca su un *POI*;
  2. l'utente seleziona un *POI* della giornata;
  3. l'utente visualizza il *POI* selezionato nella mappa;
  4. l'utente visualizza le caratteristiche del *POI* selezionato.
- **Postcondizioni**: l'utente si trova nella schermata dei poi.

UC 12.3 - Visualizzazione climb singolo

- **Attore primario**: utente autenticato;
- **Descrizione**: l'utente deve poter visualizzare un *climb* in dettaglio;
- **Precondizioni**: l'utente si trova nella schermata delle giornate;
- **Scenario principale**:
  1. l'utente clicca su un poi;
  2. l'utente seleziona un *climb* della giornata;
  3. l'utente visualizza il *climb* selezionato nella mappa;
  4. l'utente visualizza le caratteristiche del *climb* selezionato.
- **Postcondizioni**: l'utente si trova nella schermata dei climb.

UC 13 - Selezione route

- **Attore primario**: utente non autenticato;
- **Descrizione**: l'utente deve poter selezionare una route di una giornata;
- **Precondizioni**: l'utente si trova nella schermata delle giornate;
- **Scenario principale**: l'utente seleziona una route di una giornata;
- **Postcondizioni**: l'utente selezionato una route.

UC 14 - Selezione climb

- **Attore primario**: utente non autenticato;
- **Descrizione**: l'utente deve poter selezionare un climb di una giornata;
- **Precondizioni**: l'utente si trova nella schermata delle giornate;
- **Scenario principale**: l'utente seleziona un climb di una giornata;
- **Postcondizioni**: l'utente selezionato un climb.

3.2.2 Requisiti

In questa sezione vengono elencati i requisiti individuati a seguito dell'analisi delle user story, dei casi d'uso, del dialogo con il cliente e la discussione con il *project manager*. Un requisito può essere:

- **funzionale**: se indica un vincolo sugli utilizzi e funzioni del prodotto;
- **di qualità**: se indica un vincolo imposto sulle tecniche o strumenti di sviluppo;
- **di vincolo**: se indica un vincolo imposto dall'azienda o dal cliente;
- **prestazionale**: se indica un vincolo da rispettare in determinate condizioni.

Il codice di un requisito riflette il seguente formato:

`R[importanza][tipologia][numero]`

dove:

- `[importanza]` è:
  - **O**: per un requisito obbligatorio;
  - **D**: per un requisito desiderabile;
  - **P**: per un requisito opzionale.
- `[tipologia]` è:
  - **F**: per un requisito funzionale;
  - **Q**: per un requisito di qualità;
  - **V**: per un requisito di vincolo;
  - **P**: per un requisito prestazionale.
- `[numero]` è un numero progressivo univoco se il requisito non ha padre, altrimenti è nel formato:

  `[numero_padre].[numero_figlio]`

  dove:
  - `[numero_padre]` è il `[numero]` del requisito padre;
  - `[numero_figlio]` è un numero progressivo univoco tra i figli dello stesso padre.

Requisiti funzionali

Tabella 3.1: Requisiti funzionali.

Codice	Requisito	Fonte
ROF1	L'utente non autenticato deve potersi autenticare e accedere al proprio account	**UC 1**

ROF2	L'utente non autenticato deve potersi autenticare manualmente.	**UC 2**
ROF2.1	L'utente non autenticato deve poter inserire la propria email durante l'attività di autenticazione manuale.	**UC 2.1**
ROF2.2	L'utente non autenticato deve poter inserire la propria password durante l'attività du autenticazione manuale.	**UC 2.2**
ROF2.3	L'utente non autenticato deve poter richiedere la memorizzazione della sessione durante l'attività di login.	**UC 2.3**
ROF2.4	L'utente non autenticato deve poter confermare i dati inseriti e tentare di autenticarsi.	**UC 2.4**
ROF3	Il tentativo di autenticazione deve fallire nel caso in cui siano state inserite credenziali non valide.	**UC 3**
ROF4	L'utente non autenticato deve potersi autenticare automaticamente nel caso in cui abbia memorizzato la sessione precedentemente.	**UC 4**
ROF5	L'utente non autenticato deve potersi autenticare in modo provvisorio.	**UC 5**
ROF5.1	L'utente non autenticato deve poter selezionare un utente preesistente durante l'attività di autenticazione provvisoria	**UC 5.1**
ROF5.2	L'utente non autenticato deve poter confermare l'utente preesistente selezionato e tentare di autenticarsi.	**UC 5.2**
ROF6	L'utente non autenticato deve potersi registrare.	**UC 6**
ROF6.1	L'utente non autenticato deve poter inserire il proprio nome durante l'attività di registrazione.	**UC 6.1**
ROF6.2	L'utente non autenticato deve poter inserire il proprio cognome durante l'attività di registrazione.	**UC 6.2**
ROF6.3	L'utente non autenticato deve poter inserire la propria email durante l'attività di registrazione.	**UC 6.3**
ROF6.4	L'utente non autenticato deve poter inserire la propria password durante l'attività di registrazione.	**UC 6.4**
ROF6.5	L'utente non autenticato deve poter confermare la propria password durante l'attività di registrazione.	**UC 6.5**
ROF6.6	L'utente non autenticato deve poter confermare i dati inseriti e tentare di registrarsi.	**UC 6.6**
ROF7	Il tentativo di registrazione deve fallire nel caso in cui siano stati inseriti dati di registrazione non validi.	**UC 7**
ROF8	Il tentativo di registrazione deve fallire nel caso in cui l'utente creato sia già presente nel sistema.	**UC 8**
ROF9	L'utente autenticato deve poter visualizzare la mappa dell'itinerario.	**UC 9**
ROF9.1	L'utente autenticato deve poter visualizzare la sua distanza con gli altri partecipanti.	**UC 9.1**
ROF9.2	L'utente autenticato deve poter visualizzare l'accuratezza del suo segnale *GPS*.	**UC 9.2**

ROF10	L'utente autenticato deve poter ricentrare la mappa nella sua posizione.	**UC 10**
ROF11	L'utente autenticato deve modificare lo stato di condivisione della propria posizione.	**UC 11**
ROF11.1	L'utente autenticato deve poter attivare la condivisione della posizione.	**UC 11.1**
ROF11.2	l'utente autenticato deve poter disattivare la condivisione della posizione.	**UC 11.2**
ROF12	Il tentativo di attivazione della condivisione della posizione deve fallire nel caso in cui il GPS non sia attivo.	**UC 12**
ROF13	L'utente autenticato deve poter visualizzare tutte le giornate dell'itinerario corrente.	**UC 13**
ROF14	L'utente autenticato deve poter visualizzare una giornata nel dettaglio.	**UC 14**
ROF14.1	L'utente autenticato deve poter visualizzare la mappa durante l'attività di visualizzazione di una giornata nel dettaglio.	**UC 14.1**
ROF14.1.1	L'utente autenticato deve poter visualizzare i partecipanti durante l'attività di visualizzazione della mappa di una giornata.	**UC 14.1.1**
ROF14.1.2	L'utente autenticato deve poter visualizzare tutti i *POI* durante l'attività di visualizzazione della mappa di una giornata.	**UC 14.1.2**
ROF14.1.3	L'utente autenticato deve poter visualizzare la *route* percorsa di un partecipante durante l'attività di visualizzazione della mappa di una giornata.	**UC 14.1.3**
ROF14.1.4	L'utente autenticato deve poter visualizzare la *route* selezionata durante l'attività di visualizzazione della mappa di una giornata.	**UC 14.1.4**
ROF14.1.5	L'utente autenticato deve poter visualizzare il *climb* selezionato durante l'attività di visualizzazione della mappa di una giornata.	**UC 14.1.5**
ROF14.1.6	I *POI* generici e speciali devono essere visualizzati differentemente nella mappa di una giornata.	**UC 14.1.6**
ROF14.2	L'utente autenticato deve poter visualizzare i *climb* di una giornata durante l'attività di visualizzazione di una giornata nel dettaglio.	**UC 14.2**
ROF14.3	L'utente autenticato deve poter visualizzare i *poi* di una giornata durante l'attività di visualizzazione di una giornata nel dettaglio.	**UC 14.3**
ROF15	L'utente autenticato deve poter modificare la *route* selezionata in una giornata.	**UC 15**
ROF16	L'utente autenticato deve poter modificare il *climb* selezionato in una giornata.	**UC 16**

Requisiti di qualità

Tabella 3.2: Requisiti di qualità.

Codice	Requisito	Fonte
ROQ1	L'applicazione deve essere sviluppata con in *dart*.	**Azienda**
ROQ2	L'applicazione deve essere sviluppata con il *framework flutter*.	**Azienda**
RDQ2.1	l'applicazione *flutter* deve essere sviluppata con il *bloc pattern*.	**Azienda**
ROQ3	I sorgenti devono essere versionati con lo strumento *git*.	**Azienda**
ROQ4	Il repository remoto per il versionamento destinato all'azienda deve essere *BitBucket*.	**Azienda**
ROQ5	Il repository remoto per il versionamento destinato al cliente deve essere *GitHub*.	**Cliente**
ROQ6	Il *back-end* dell'applicazione deve esporre un *API* REST.	**Cliente**
RDQ7	La base di dati per il salvataggio in locale dei dati deve essere *Sqlite*.	**Cliente**

Requisiti di vincolo

Tabella 3.3: Requisiti di vincolo.

Codice	Requisito	Fonte
ROV1	La posizione dell'utente dell'applicazione deve essere riconoscibile.	**Cliente**
ROV2	La posizione del pulmino di soccorso deve essere riconoscibile.	**Cliente**
ROV3	La giornata corrente di un itinerario deve essere evidenziata.	**Cliente**
RDV4	Se l'utente ha ricentrato la sua posizione, la mappa deve ricentrarsi automaticamente nel caso in cui l'utente aggiorni la sua posizione.	**Azienda**
RDV5	Il tratto di *route* percorsa deve essere visualizzata nella mappa.	**Azienda**
ROV6	I *POI* normali devono essere visulizzati differentemente da quelli speciali.	**Cliente**
ROV7	L'interfaccia dell'applicazione deve seguire il più fedelmente possibile i *mockup* dell'applicazione.	**Cliente**
ROV8	La condivisione della posizione deve poter funzionare anche in *background*.	**Cliente**

Requisiti prestazionali

Tabella 3.4: Requisiti prestazionali.

Tabella dei requisiti		
Codice	**Requisito**	**Fonte**

ROP1	L'invio della propria posizione deve avvenire solo quando l'utente si muove di almento cinque metri	**Azienda**
ROP2	La ricezione delle posizioni del gruppo deve avvenire ogni trenta secondi	**Cliente**
RDP3	I *GPX* non devono essere richiesti più di una volta	**Azienda**

3.2.3 Tracciamento

Fonte - Requisiti

Tabella 3.5: Tracciamento fonte - requisiti.

Fonte	**Requisito**
Azienda	**ROQ1** **ROQ2** **RDQ2.1** **ROQ3** **ROQ4** **ROQ5** **ROQ6** **RDQ7** **RDV4** **RDV5** **ROP1** **RDP3**
Cliente	**ROQ5** **ROQ6** **RDQ7** **ROV1** **ROV2** **ROV3** **ROV6** **ROV7** **ROV8** **ROP2**
UC 1	**ROF1**
UC 2	**ROF2**
UC 2.1	**ROF2.1**
UC 2.2	**ROF2.2**
UC 2.3	**ROF2.3**
UC 2.4	**ROF2.4**
UC 3	**ROF3**
UC 4	**ROF4**
UC 5	**ROF5**
UC 5.1	**ROF5.1**
UC 5.2	**ROF5.2**
UC 6	**ROF6**
UC 6.1	**ROF6.1**

UC 6.2	**ROF6.2**
UC 6.3	**ROF6.3**
UC 6.4	**ROF6.4**
UC 6.5	**ROF6.5**
UC 6.6	**ROF6.6**
UC 7	**ROF7**
UC 8	**ROF8**
UC 9	**ROF9**
UC 9.1	**ROF9.1**
UC 9.2	**ROF9.2**
UC 10	**ROF10**
UC 11	**ROF11**
UC 11.1	**ROF11.1**
UC 11.2	**ROF11.2**
UC 12	**ROF12**
UC 13	**ROF13**
UC 14	**ROF14**
UC 14.1	**ROF14.1**
UC 14.1.1	**ROF14.1.1**
UC 14.1.2	**ROF14.1.2**
UC 14.1.3	**ROF14.1.3**
UC 14.1.4	**ROF14.1.4**
UC 14.1.5	**ROF14.1.5**
UC 14.1.6	**ROF14.1.6**
UC 14.2	**ROF14.2**
UC 14.3	**ROF14.3**
UC 15	**ROF15**
UC 16	**ROF16**

Requisiti - Fonte

Tabella 3.6: Tracciamento requisiti - fonte.

Requisito	**Fonte**
ROF1	**UC 1**
ROF2	**UC 2**
ROF2.1	**UC 2.1**
ROF2.2	**UC 2.2**
ROF2.3	**UC 2.3**
ROF2.4	**UC 2.4**
ROF3	**UC 3**
ROF4	**UC 4**
ROF5	**UC 5**
ROF5.1	**UC 5.1**
ROF5.2	**UC 5.2**
ROF6	**UC 6**
ROF6.1	**UC 6.1**

ROF6.2	**UC 6.2**
ROF6.3	**UC 6.3**
ROF6.4	**UC 6.4**
ROF6.5	**UC 6.5**
ROF6.6	**UC 6.6**
ROF7	**UC 7**
ROF8	**UC 8**
ROF9	**UC 9**
ROF9.1	**UC 9.1**
ROF9.2	**UC 9.2**
ROF10	**UC 10**
ROF11	**UC 11**
ROF11.1	**UC 11.1**
ROF11.2	**UC 11.2**
ROF12	**UC 12**
ROF13	**UC 13**
ROF14	**UC 14**
ROF14.1	**UC 14.1**
ROF14.1.1	**UC 14.1.1**
ROF14.1.2	**UC 14.1.2**
ROF14.1.3	**UC 14.1.3**
ROF14.1.4	**UC 14.1.4**
ROF14.1.5	**UC 14.1.5**
ROF14.1.6	**UC 14.1.6**
ROF14.2	**UC 14.2**
ROF14.3	**UC 14.3**
ROF15	**UC 15**
ROF16	**UC 16**
ROQ1	**Azienda**
ROQ2	**Azienda**
RDQ2.1	**Azienda**
ROQ3	**Azienda**
ROQ4	**Azienda**
ROQ5	**Cliente**
ROQ6	**Cliente**
RDQ7	**Cliente**
ROP1	**Cliente**
ROP2	**Cliente**
ROP3	**Cliente**
RDF4	**Azienda**
RDF5	**Azienda**
ROF6	**Cliente**
ROF7	**Cliente**
ROF8	**Cliente**
ROP1	**Azienda**
ROP2	**Cliente**
RDP3	**Azienda**

Riepilogo

La tabella 3.7 presenta il riepilogo dei requisiti.

Tabella 3.7: Riepilogo dei requisiti.

Tipologia	**Obbligatori**	**Opzionali**	**Desiderabili**	**Totale**
Funzionali	40	0	0	40
Di qualità	6	0	2	8
Di vincolo	6	0	2	8
Prestazionali	2	0	1	3
Totale	54	0	5	59

3.3 Progettazione

3.3.1 Architettura di sistema

L'architettura segue un normale *modello client-server*, nella quale l'applicazione flutter svolge il ruolo di client e il *back-end* offerto dal cliente svolge quello di server. Tralasciando l'architettura del *back-end*, andiamo ad analizzare l'archittura dell'applicazione da me sviuluppata.

Architettura dell'applicazione

L'architettura dell'applicazione è illustrata nella figura 3.11, questa si basa in larga parte al modello presentato in §2.4. Sono presenti due package principali, *Presentation* e *Data*, il primo si occupa di tutto ciò che concerne l'interazione con l'utente, quindi la visualizzaione delle schermate e gli eventi, il secondo si occupa di tutto ciò che concerne i dati di dominio dell'applicazione, fornendo punti di accesso e operazioni su di essi. Il terzo package *Utils* si occupa solo di mantenere delle classi ausiliarie e non rappresenta un fattore architetturale di rilievo.

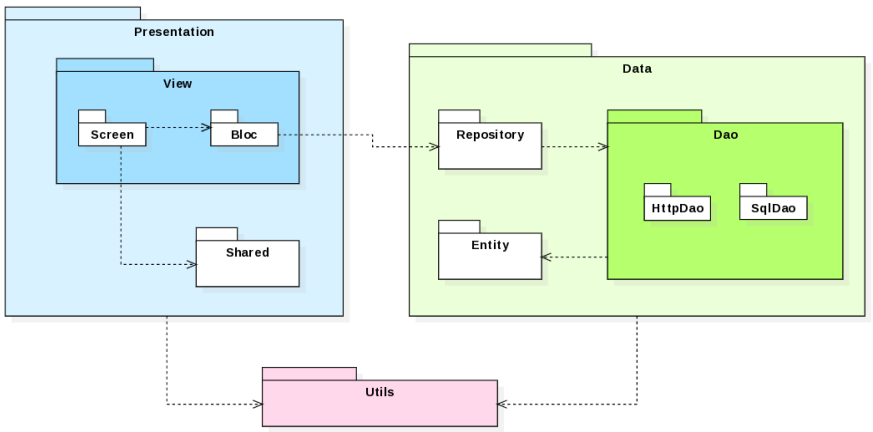

Figura 3.11: Architettura di CyclingHero.

Descrizione delle componenti

Dall'architettura di sistema si individuano le principali componenti, questa sezione si occupa di fornire una breve descrizione sulle loro funzioni.

Presentation Si occupa di costruire la parte grafica dell'applicazione e gestire l'interazione con l'utente, si compone di:

- **View**: definisce le schermate dell'applicazione, si compone di:
  - **Screen**: definisce una schermata in modo strutturale sulla base dello stato dal relativo *bloc*;
  - **Bloc**: definisce gli aspetti comportamentali di una schermata, tramite eventi e stati, comunica direttamente con i *repository*.
- **Shared**: definisce gli elementi comuni per ogni schermata.

Data Si occupa di reperire e manipolare i dati dell'applicazione, si compone di;

- **Repository**: definisce l'interfaccia di accesso ai dati, comunica direttamente con i *Dao*;
- **Dao**: fornisce l'implementazione per l'accesso ai dati in diverse sorgenti, nel caso dell'applicazione a quelle HTTP e *SQL*;
- **Entity**: definisce le entità partecipanti nel sistema.

3.4 Codifica

In questa sezione vengono presentate le principali schermate dell'applicazione, per ogni schermata viene elaborata una descrizione sintetica e una lista di funzionalità implementate, successivamente viene analizzato in dettaglio il lato tecnico di tale interfaccia, accompagnando il tutto con dei diagrammi di sequenza per le attività rilevanti.

3.4.1 Gestione dell'autenticazione

In questa sezione vengono analizzate le schermate di autenticazione manuale e provvisoria, le altre schermate inerenti all'autenticazione, cioè quella di registrazione e recupero password sono analoghe a quella di autenticazione manuale, perciò non verranno approfondite.

Descrizione

CyclingHero permette l'autenticazione per due differenti vie: quella manuale e quella provvisoria (figura 3.12) . La schermata di autenticazione manuale permette all'utente di collegarsi all'applicazione inserendo le sue credenziali, mentre quella provvisoria consente l'autenticazione tramite la scelta di un utente preesistente. La schermata di autenticazione provvisoria è stata richiesta dal cliente in quanto *back-end* non disponeva ancora di un *API* per l'autenticazione.

Interfaccia

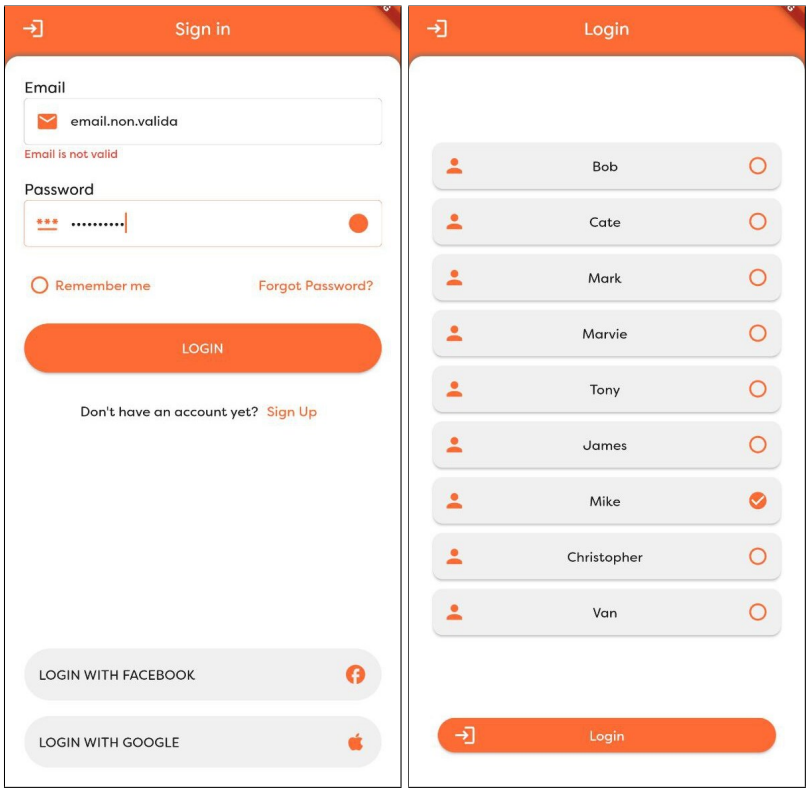

Figura 3.12: Da sinistra verso destra: schermata di autenticazione manuale, schermata di autenticazione provvisoria.

Implementazione

Schermata di autenticazione manuale La schermata di autenticazione manuale è basata fondamentalmente sulle seguenti classi:

- **LoginScreen**: determina l'aspetto grafico della schermata, qui viene definita la gerarchia di *widget* che compone la schermata. I principali *widget* utilizzati sono: *InputFieldCh* per l'acquisizione dei dati e*ButtonCh* per i pulsanti di autenticazione;
- **LoginBloc**: rappresenta l'aspetto logico della schermata e svolge tre importanti compiti:
  - definisce tutti gli eventi inerenti alla schermata, tra cui *SubmitLoginEvent* per indicare l'azione di autenticazione e *InputChangedEvent* per indicare la modifica di un campo del modulo;
  - definisce lo stato della schermata, contiene gli *InputField* e lo stato di autenticazione;
  - gestisce gli eventi in arrivo interfacciandosi con lo strato sottostante con il *UserRepository* per gestire l'autenticazione.

- **InputField**: è una classe da me concepita per astrarre il concetto di campo di *input*, questa classe svolge tre compiti:
  - mantiene il valore dell'input rappresentato;
  - mantiene lo stato di validazione dell'intput.

Nella figura 3.13 viene mostrato il diagramma di sequenza per processo di autenticazione. Per prima cosa l'evento *SubmitLoginEvent* viene lanciato dalla schermata, *LoginBloc*, dopo aver ricevuto l'evento e verificato la validità degli input, aggiorna il suo stato interno indicando un attesa di autenticazione che *LoginScreen* interpreterà con la visualizzazione di un messaggio di attesa. Successivamente, *LoginBloc* relega l'attività di autenticazione a *UserRepository* che allo stesso modo incarica *UserHttpDao* per finalizzare la richiesta all'*API*. In caso di successo, *UserRepository* si occuperà di gestire il contesto di autenticazione per l'applicazione e successivamente invierà a *LoginBloc* l'esito dell'operazione, in quel momento *LoginBloc* aggiornarà lo stato di successo di autenticazione, che si ripercuoterà nella schermata di autenticazione visualizzando un messaggio di successo.

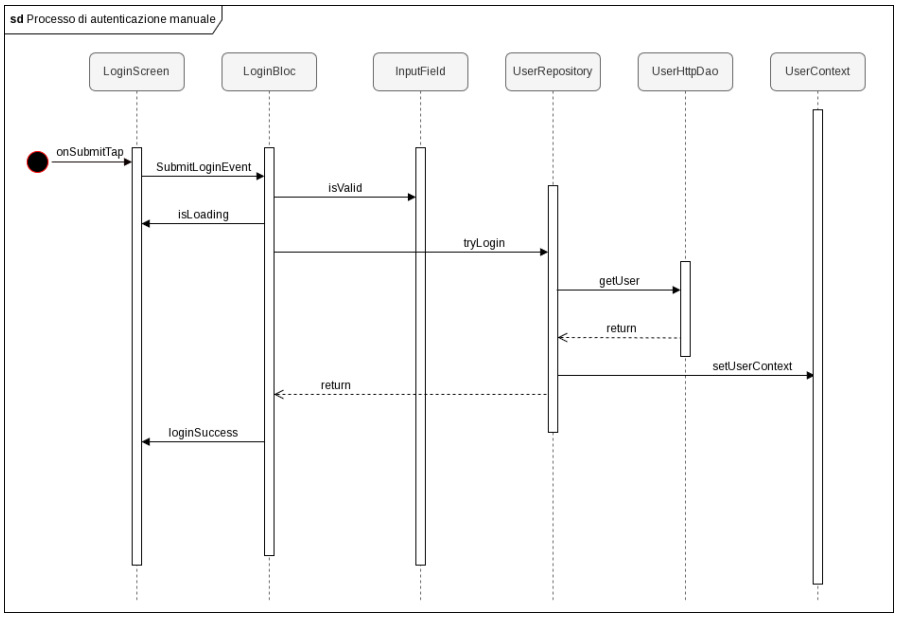

Figura 3.13: Diagramma di sequenza per il processo di autenticazione.

Schermata di autenticazione manuale La schermata di autenticazione provvisoria è stata un esigenza del cliente per ovviare la mancanza degli *endpoint* di autenticazione. Come illustrato nel diagramma di sequenza della figura 3.14, l'autenticazione avviene semplicemente selezionando uno degli utenti preesistenti.

Resoconto dei requisiti associati

La tabella 3.8 elenca i requisiti associati all'attività di autenticazione. Sebbene nell'applicazione le funzionalità per l'autenticazione siano presenti, la mancanza del relativo

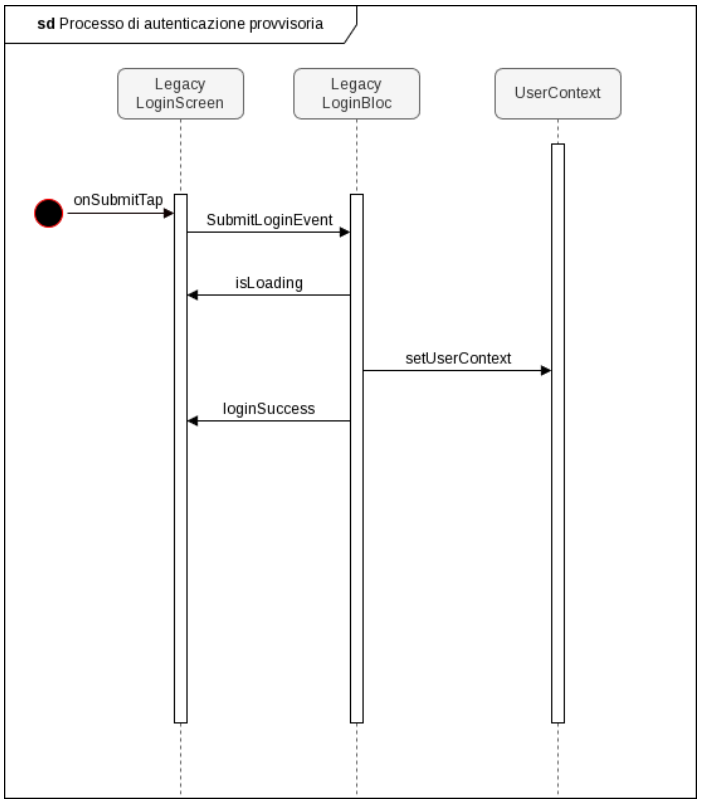

Figura 3.14: Diagramma di sequenza per il processo di autenticazione provvisoria.

endpoint del *back-end* non permette effettivamente l'autenticazione, perciò non sono riuscito a soddisfare tutti i requisiti di autenticazione.

Tabella 3.8: Soddisfacimento dei requisiti associati alla gestione dell'autenticazione.

Codice	Descrizione	Stato
ROF1	L'utente non autenticato deve potersi autenticare e accedere al proprio account	**Soddisfatto**
ROF2	L'utente non autenticato deve potersi autenticare manualmente.	**Non soddisfatto**
ROF2.1	L'utente non autenticato deve poter inserire la propria email durante l'attività di autenticazione manuale.	**Soddisfatto**
ROF2.2	L'utente non autenticato deve poter inserire la propria password durante l'attività du autenticazione manuale.	**Soddisfatto**
ROF2.3	L'utente non autenticato deve poter richiedere la memorizzazione della sessione durante l'attività di login.	**Soddisfatto**
ROF2.4	L'utente non autenticato deve poter confermare i dati inseriti e tentare di autenticarsi.	**Non soddisfatto**

ROF3	Il tentativo di autenticazione deve fallire nel caso in cui siano state inserite credenziali non valide.	**Soddisfatto**
ROF4	L'utente non autenticato deve potersi autenticare automaticamente nel caso in cui abbia memorizzato la sessione precedentemente.	**Non soddisfatto**
ROF5	L'utente non autenticato deve potersi autenticare in modo provvisorio.	**Soddisfatto**
ROF5.1	L'utente non autenticato deve poter selezionare un utente preesistente durante l'attività di autenticazione provvisoria	**Soddisfatto**
ROF5.2	L'utente non autenticato deve poter confermare l'utente preesistente selezionato e tentare di autenticarsi.	**Soddisfatto**
ROF6	L'utente non autenticato deve potersi registrare.	**Non soddisfatto**
ROF6.1	L'utente non autenticato deve poter inserire il proprio nome durante l'attività di registrazione.	**Soddisfatto**
ROF6.2	L'utente non autenticato deve poter inserire il proprio cognome durante l'attività di registrazione.	**Soddisfatto**
ROF6.3	L'utente non autenticato deve poter inserire la propria email durante l'attività di registrazione.	**Soddisfatto**
ROF6.4	L'utente non autenticato deve poter inserire la propria password durante l'attività di registrazione.	**Soddisfatto**
ROF6.5	L'utente non autenticato deve poter confermare la propria password durante l'attività di registrazione.	**Soddisfatto**
ROF6.6	L'utente non autenticato deve poter confermare i dati inseriti e tentare di registrarsi.	**Non soddisfatto**
ROF7	Il tentativo di registrazione deve fallire nel caso in cui siano stati inseriti dati di registrazione non validi.	**Soddisfatto**
ROF8	Il tentativo di registrazione deve fallire nel caso in cui l'utente creato sia già presente nel sistema.	**Non soddisfatto**

3.4.2 Gestione della mappa del gruppo

Descrizione

La schermata di condivisione della posizioni (figura 3.15) consente all'utente di condividere e ricevere le posizioni del gruppo, offre inoltre un meccanismo per ricentrare la mappa in modo intelligente. Infine consente di visualizzare il gruppo e la propria distanza da ogni partecipante.

Interfaccia

Figura 3.15: Schermata della mappa principale del gruppo.

Implementazione

La schermata principale della mappa del gruppo è basata fondamentalmente sulle seguenti classi:

- **LocationScreen**: determina l'aspetto grafico della schermata, qui viene definita la gerarchia di *widget* che compone la schermata. I dati per la visualizzazione degli utenti sono contenuti nel *LocationBloc*. I principali *widget* utilizzati sono: *GoogleMapCh* per visualizzare la mappa e disegnare gli utenti, *ButtonCh* per i pulsanti di condivisione e ricentramento e *PanelCh* per il pannello a scorrimento contenente le impostazioni di condivisione;

- **LocationBloc**: rappresenta l'aspetto logico della schermata e svolge tre importanti compiti:

  – definisce tutti gli eventi inerenti alla schermata, tra cui *ToggleSharingEvent* per attivare o disattivare la condivisione della posizione, *LoadGroupEvent* per ricevere le posizioni degli altri partecipanti e *CenterMapEvent* per centrare la mappa nella posizione dell'utente;

  – definisce lo stato della schermata salvandosi le posizioni dei partecipanti;

  - gestisce gli eventi in arrivo interfacciandosi con lo strato sottostante con il *UserRepository* per gestire la ricezione e l'invio dei dati.

- **LocationUtils**: è una classe da me concepita per semplificare l'*API* di accesso alla posizione del dispositivo.

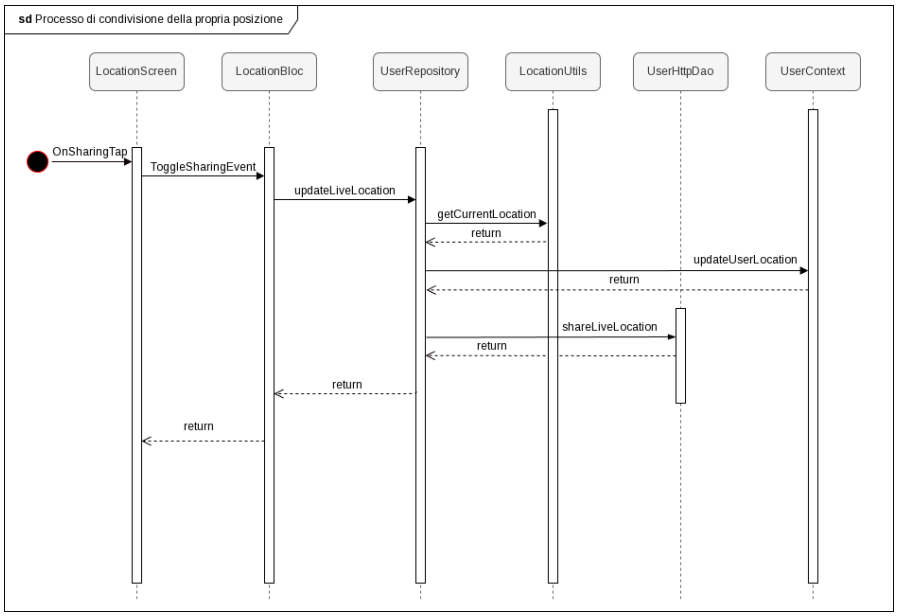

Figura 3.16: Diagramma di sequenza per il processo di condivisione della posizione.

Il diagramma di sequenza in figura 3.16 mostra il normale processo di condivisione della posizione: dopo aver ricevunto l'evento di attivazione della posizione, *LocationBloc* invia una richiesta al *userRepository* per avviare la condivisione. Per far questo *UserRepository* inizia a rimanere in ascolto delle variazioni della posizione del dispositivo, se *LocationUtils* rileva una variazione della posizione superiore di cinque metri rispetto alla precedente, allora informa il *UserRepository* del nuovo dato, che passerà al *UserHttpDao* per condivederla con il *back-end*. Il processo nel suo complesso non è bloccnte per il *LocationScreen*, che dopo il primo evento emesso in modo asincrono ritorna normalmente operativa.

Resoconto dei requisiti associati

La tabella 3.9 elenca i requisiti associati all'attività di autenticazione. Ogni requisito identificato è stato soddisfatto e validato dal cliente.

Tabella 3.9: Soddisfacimento dei requisiti associati alla gestione della mappa del gruppo.

Codice	Descrizione	Stato
ROF9	L'utente autenticato deve poter visualizzare la mappa dell'itinerario.	**Soddisfatto**
ROF9.1	L'utente autenticato deve poter visualizzare la sua distanza con gli altri partecipanti.	**Soddisfatto**

ROF9.2	L'utente autenticato deve poter visualizzare l'accuratezza del suo segnale *GPS*.	**Soddisfatto**
ROF10	L'utente autenticato deve poter ricentrare la mappa nella sua posizione.	**Soddisfatto**
ROF11	L'utente autenticato deve modificare lo stato di condivisione della propria posizione.	**Soddisfatto**
ROF11.1	L'utente autenticato deve poter attivare la condivisione della posizione.	**Soddisfatto**
ROF11.2	L'utente autenticato deve poter disattivare la condivisione della posizione.	**Soddisfatto**
ROF12	Il tentativo di attivazione della condivisione della posizione deve fallire nel caso in cui il GPS non sia attivo.	**Soddisfatto**
ROV1	La posizione dell'utente dell'applicazione deve essere riconoscibile.	**Soddisfatto**
ROV2	La posizione del pulmino di soccorso deve essere riconoscibile.	**Soddisfatto**
RDV4	Se l'utente ha ricentrato la sua posizione, la mappa deve ricentrarsi automaticamente nel caso in cui l'utente aggiorni la sua posizione.	**Soddisfatto**
ROV8	La condivisione della posizione deve poter funzionare anche in *background*.	**Soddisfatto**
ROP1	L'invio della propria posizione deve avvenire solo quando l'utente si muove di almento cinque metri	**Soddisfatto**
ROP2	la ricezione delle posizioni del gruppo deve avvenire ogni trenta secondi	**Soddisfatto**

3.4.3 Gestione dell'itinerario

Descrizione

La gestione dell'itinerario (figura 3.17) è un processo implementato da più schermate:

- **schermata dell'itinerario corrente**: visualizza un elenco delle giornate dell'itinerario e delle loro caratteristiche, quali: data, luogo di partenza e luogo di arrivo;
- **schermata di una giornata in dettaglio**: visualizza tutte le informazioni relative a una giornata, consente di visualizzare una mappa contenente le *route* e i *climb* della giornata, visualizzare la *route* percorsa e selezionare diverse *route*;
- **schermata di un *climb* o *POI* in dettaglio**: visualizza tutte le informazioni di un *climb* o di un *POI*.

La prima e la terza schermata sono relativamente semplici e non verranno analizzate in questa sezione.

Interfaccia

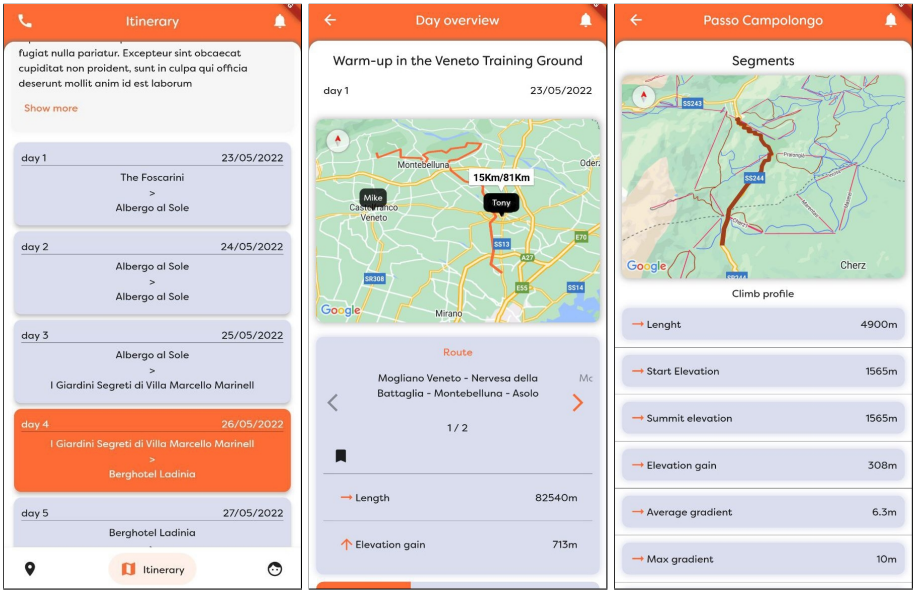

Figura 3.17: Da sinistra verso destra: schermata delle giornate di un *tour*, schermata di una giornata in dettaglio, schermata di un *climb* in dettaglio.

Implementazione

La schermata di una giornata in dettaglio è basata fondamentalmente sulle seguenti classi:

- **DayScreen**: determina l'aspetto grafico della schermata, qui viene definita la gerarchia di *widget* che compone la schermata. Visualizza una mappa della *route* selezionata, un carosello per modificare la selezione, un carosello per visualizzare i *POI* della *route* e altre informazioni minori sulla giornata. I principali *widget* utilizzati sono: *GoogleMapCh* per: visualizzare la mappa, disegnare gli utenti, disegnare la *route* percorsa, disegnare i *climb* e *Swiper* creare i caroselli;

- **DayBloc**: rappresenta l'aspetto logico della schermata e svolge tre importanti compiti:

  – definisce tutti gli eventi inerenti alla schermata, tra cui *SwitchRouteEvent* e *SwitchClimbEvent* per selezionare rispettivamente una nuova *route* e un nuovo *climb*;

  – definisce lo stato della schermata, mantiene la *route* selezionata, *climb* selezionato e i relativi GPX;

  – gestisce gli eventi in arrivo interfacciandosi con lo strato sottostante, con il *RouteRepository* e il *ClimbRepository* per l'ottenimento dei *GPX*.

- **RouteSqlDao**: è una classe da me concepita implementare il salvataggio in locale dei *GPX*, evitando quindi di dover richiedere al *back-end* dati già ricevuti.

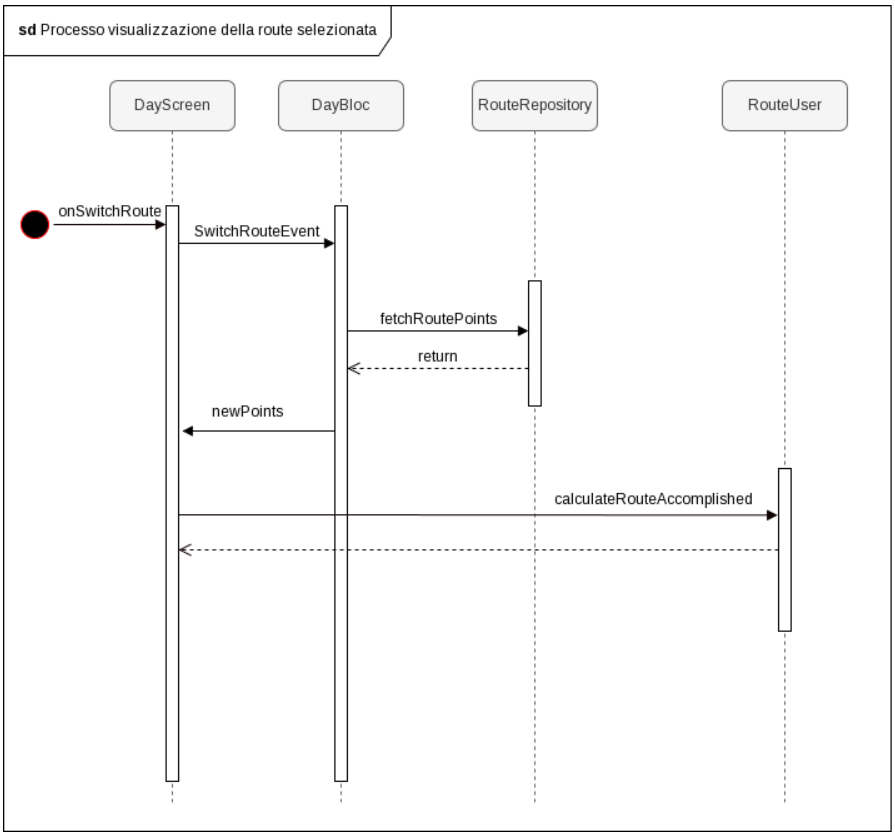

Figura 3.18: Diagramma di sequenza per il processo di calcolo della *route percorsa*.

Il diagramma di sequenza in figura 3.18 mostra il normale processo di selezione di una nuova *route* visualizzata: per prima cosa *DayBloc* richiede al *RouteRepository* i dati necessari, a quel punto il *repository* applica una semplice politica di selezione, restituisce il *GPX* in locale nel caso sia stato ricevuto in precedenza, altrimenti esegue una nuova richiesta al *RouteHttpDao* per ottenere i dati e salvarli in locale. Dopo aver ricevuto i dati, *DayBloc* li fornisce al *DayScreen*, il quale tramite la posizione dell'utente corrente esegue il calcolo della *route* percorsa.

Algoritmo di calcolo della *route* percorsa Il calcolo della *route* percorsa è un semplice algoritmo che data una posizione e un insieme di punti geografici, esegue una ricerca per identificare il punto di minor distanza dalla posizione interessata. Dal punto ottenuto viene calcolata la distanza totale percorsa sommando cumulativamente le distanze dei punti precedenti[1].

Resoconto dei requisiti associati

La tabella 3.10 elenca i requisiti associati alla gestione dell'itinerario. Ogni requisito identificato è stato soddisfatto e validato dal cliente.

[1] *Il formato GPX*. URL: `https://www.topografix.com/gpx.asp`.

Tabella 3.10: Soddisfacimento dei requisiti associati alla gestione dell'itinerario.

Codice	Descrizione	Stato
ROF9	L'utente autenticato deve poter visualizzare la mappa dell'itinerario.	**Soddisfatto**
ROF9.1	L'utente autenticato deve poter visualizzare la sua distanza con gli altri partecipanti.	**Soddisfatto**
ROF9.2	L'utente autenticato deve poter visualizzare l'accuratezza del suo segnale *GPS*.	**Soddisfatto**
ROF10	L'utente autenticato deve poter ricentrare la mappa nella sua posizione.	**Soddisfatto**
ROF11	L'utente autenticato deve modificare lo stato di condivisione della propria posizione.	**Soddisfatto**
ROF11.1	L'utente autenticato deve poter attivare la condivisione della posizione.	**Soddisfatto**
ROF11.2	l'utente autenticato deve poter disattivare la condivisione della posizione.	**Soddisfatto**
ROF12	Il tentativo di attivazione della condivisione della posizione deve fallire nel caso in cui il GPS non sia attivo.	**Soddisfatto**
ROF13	L'utente autenticato deve poter visualizzare tutte le giornate dell'itinerario corrente.	**Soddisfatto**
ROF14	L'utente autenticato deve poter visualizzare una giornata nel dettaglio.	**Soddisfatto4**
ROF14.1	L'utente autenticato deve poter visualizzare la mappa durante l'attività di visualizzazione di una giornata nel dettaglio.	**Soddisfatto**
ROF14.1.1	L'utente autenticato deve poter visualizzare i partecipanti durante l'attività di visualizzazione della mappa di una giornata.	**Soddisfatto**
ROF14.1.2	L'utente autenticato deve poter visualizzare tutti i *POI* durante l'attività di visualizzazione della mappa di una giornata.	**Soddisfatto**
ROF14.1.3	L'utente autenticato deve poter visualizzare la *route* percorsa di un partecipante durante l'attività di visualizzazione della mappa di una giornata.	**Soddisfatto**
ROF14.1.4	L'utente autenticato deve poter visualizzare la *route* selezionata durante l'attività di visualizzazione della mappa di una giornata.	**Soddisfatto**
ROF14.1.5	L'utente autenticato deve poter visualizzare il *climb* selezionato durante l'attività di visualizzazione della mappa di una giornata.	**Soddisfatto**
ROF14.1.6	I *POI* generici e speciali devono essere visualizzati differentemente nella mappa di una giornata.	**Soddisfatto**
ROF14.2	L'utente autenticato deve poter visualizzare i *climb* di una giornata durante l'attività di visualizzazione di una giornata nel dettaglio.	**Soddisfatto**

ROF14.3	L'utente autenticato deve poter visualizzare i *poi* di una giornata durante l'attività di visualizzazione di una giornata nel dettaglio.	**Soddisfatto**
ROF15	L'utente autenticato deve poter modificare la *route* selezionata in una giornata.	**Soddisfatto**
ROF16	L'utente autenticato deve poter modificare il *climb* selezionato in una giornata.	**Soddisfatto**
ROV3	La giornata corrente di un itinerario deve essere evidenziata.	**Soddisfatto**
RDV5	Il tratto di *route* percorsa deve essere visualizzata nella mappa.	**Soddisfatto**
ROV6	I *POI* normali devono essere visulizzati differentemente da quelli speciali.	**Soddisfatto**
RDP3	I *GPX* non devono essere richiesti più di una volta	**Soddisfatto**

3.5 Verifica e validazione

In questa sezione viene descritto i processi di verifica e validazione attuati durante il progetto. Con verifica si intendono tutte le pratiche adottate al fine di prevenire l'introduzione di errori all'interno del prodotto, mentre con validazione si intende il processo di accertamento dei requisiti. Per quanto riguarda la verifica, per i *test* statici mi sono avvalso del linter ufficiale di *Flutter*, il quale va a eseguire un'analisi statica del codice seguendo determinate regole preconfigurate[2], ciò mi ha permesso di applicare uno stile di codifica consistente in tutto il progetto ed evitare eventuali errori sintattici. I *test* dinamici sono stati fatti manualmente da me con l'aiuto del debugger di *VSCode*, non sono riuscito a creare una *suit* di *testing* per automatizzare la verifica in quanto le brevi tempistiche di rilascio non mi hanno permesso di concentrarmi in attività secondarie allo sviluppo. Per quanto riguarda la validazione, questa è stata perseguita periodicamente insieme al *project manager* prima di ogni rilascio, verificando il soddisfacimento degli obiettivi prefissati nel *sprint planning*.

[2] *Dart linter*. URL: `https://dart.dev/tools/linter-rules`.

Capitolo 4

Conclusione

In questo capitolo viene attuata una valutazione retrospettiva del tirocinio svolto, andando a stilare gli obiettivi raggiunti e le conoscenze acquisite. Viene infine ritagliato uno spazio per le considerazioni personali sull'esperienza del tirocinio.

4.1 Obiettivi raggiunti

Al completamento del tirocinio, insieme al *project manager* e al *tutor* aziendale, abbiamo verificato il soddisfacimento degli obiettivi stabiliti. Quasi tutti tutti gli obiettivi sono stati raggiunti, in particolare tutti gli obiettivi obbligatori sono stati rispettati ed è stato soddisfatto anche l'unico obiettivo desiderabile, andando a implementare funzioni inizialmente non previste, come il calcolo delle distanze tra i partecipanti, il *caching* dei dati e la visualizzazione della *route* percorsa. La tabella 4.1 offre un riepilogo dello stato degli obiettivi alla conclusione del tirocinio.

Tabella 4.1: Completamento degli obiettivi del tirocinio.

Codice	Descrizione	Stato
OO1	La gestione e la pianificazione del progetto deve essere gestita tramite una *board kanban* condivisa.	raggiunto
OO2	Il tirocinante deve praticare un'analisi e una progettazione dell'applicazione *mobile* a partire dai requisiti raccolti.	raggiunto
OO3	Il tirocinante deve sviluppare il modulo di autenticazione dell'applicazione.	raggiunto
OO4	Il tirocinante deve sviluppare il modulo di raccolta e invio dei dati *gps*.	raggiunto
OO5	Il tirocinante deve sviluppare il modulo per la gestione dei *tour*, *route* e *POI*.	raggiunto
OD1	Il tirocinante deve essere in grado di coordinarsi con il cliente.	raggiunto
OD2	Il tirocinante deve sviluppare la *suit* di *testing* del prodotto.	non raggiunto

OD3	Il tirocinante deve redarre una documentazione completa del prodotto.	raggiunto
OP1	Ulteriori modifiche al prodotto non comprese negli obiettivi obbligatori.	raggiunto

4.2 Conoscenze acquisite

Il periodo di tirocinio mi ha permesso di apprendere nuovi processi di sviluppo, a me prima sconosciuti. Partendo dall'inizio dell'esperienza: durante le prime settimane ho acquisito una buona dimetichezza con il linguaggio *Dart* e il *framework Flutter*, contemporaneamente, sono stato immerso nella metodologia di lavoro agile, di cui non conoscevo le modalità prima di allora.
Nelle settimane successive ho raffinato le conoscenze tecniche andando nel dettaglio di nuovi *pattern* architetturali e qui ho appreso per la prima volta il *pattern bloc*, che nel progetto ho poi usato estensivamente. Lavorare su un'applicazione multipiattaforma mi ha poi introdotto nello specifico agli ambienti *iOS* e *Android*, con la quale mi sono interfacciato per il processo di rilascio dell'applicativo.
Infine, dal progetto in sè ho assimilato concetti specifici, come l'utilizzo di *API* di mappe geografiche, interfacciamento ai sistemi *GPS* e l'utilizzo di formati *GPX*.

4.3 Considerazioni

4.3.1 L'azienda

L'esperienza con *Moku S.r.l.*, l'azienda ospitante, è stata sicuramente positiva Durante i due mesi di tirocinio il *team* è rimasto sempre dispobibile a qualsiasi chiarimento e dubbio. L'unica difficoltà che posso dire di aver incontrato è stato lo studio di *flutter*, infatti la tecnologia in questione era nuova per il *tean*, quindi l'apprendimento iniziale è stato a mio parere più lungo di quanto sarebbe potuto essere nel caso in cui ci fosse stato un esperto a cui rivolgermi.

4.3.2 Il cliente

L'esperienza con il cliente è stata una successione di alti e bassi, sebbene il cliente fosse sempre disponibile a eventuali chiarimenti e discussioni, *CyclingHero* è stato un definito troppo velocemente e questo a causato diversi rallentamenti: in primo luogo i requisiti sono stati più volte rivisitati dal cliente, creando inutili perdite di tempo, in secondo la tempistica per i rilasci è stata troppo frettolosa, incentivando uno sviluppo poco solido, volto solamente a rispettare le scadenze. Infine, al termine del tirocinio il *back-end* offerto dal cliente si trovava ancora in una stadio preliminare, ciò mi ha impedito di implementare diverse funzionalità, in particolare quella di autenticazione, propedeutica a molte altre funzionalità.

4.3.3 Commento finale

L'esperienza del tirocinio mi portato a una crescita su più livelli, che oltre alle conoscenze tecniche acquisite mi ha permesso di inquadrare per la prima volta un'ambiente di lavoro reale, attraverso interazioni tra colleghi, *project manager* e clienti esterni.

Acronimi e abbreviazioni

API Application Program Interface. 57

BLOC Business LOgic Component. 8

GPS Global Positioning System. 58

GPX GPS Exchange Format. 58

HTTP HyperText Transfer Protocol. 58

JSON JavaScript Object Notation. 58

POI Point Of Interest. 58

REST REpresentational State Transfer. 58

UML Unified Modeling Language. 59

Glossario

Agile approccio di gestione di progetto e sviluppo che aiuta il *team* a rilasciare un prodotto velocemente, rilasciando incrementalmente e in breve tempo versioni parziali del prodotto, in modo da avere un riscontro nel breve termine per aggiornare e raffinare i requisiti identificati. 1

API insieme di procedure che offrono servizi per lo sviluppo e integrazione di sistemi software diversi. La finalità è ottenere un'astrazione, di solito tra l'hardware e il programmatore o tra software a basso e quello ad alto livello semplificando così il lavoro di programmazione. 8, 55

Attore Entità, umana o meno, che interagisce con il sistema. 11

Back-end in un *modello client-server* il back-end rappresenta il *server*. Tra processi di *back-end* ci sono l'accesso ai dati e l'elaborazione delle richieste. 2

kanban sistema di rappresentazione grafico del lavoro nelle varie fasi di un processo, utilizza delle schede per rappresentare delle attività e le colonne per rappresentare ogni fase del processo. Le schede vengono spostate da sinistra a destra per mostrare i progressi. 3

Caso d'uso insieme di scenari (sequenze di azioni) che hanno in comune uno scopo finale (obiettivo) per un *attore*. 11

Climb nel contesto applicativo indica un particolare tratto della *route* delimitato da un punto di inizio e uno di fine, di cui si sanno determinate caratteristiche, come la ripidità e la difficoltà di salita. 2

Debugger strumento con lo specifico obiettivo di testare altri programmi, solitamente si occupa di lanciare un programma in un ambiente controllato, per permettere al programmatore di analizzare l'esecuzione del programma. 51

Flutter framework creato da *Google* per la creazione di applicazioni multipiattaforma. 7

Framework Architettura logica di supporto sulla quale un software può essere progettato e realizzato, facilitandone lo sviluppo. 5, 57

Frontend rappresenta tutto ciò che concerne la parte di presentazione dei dati all'utente. 2

GPS sistema di posizionamento e navigazione satellitare militare statunitense, attraverso una rete dedicata di satelliti artificiali in orbita, fornisce a un terminale mobile o ricevitore GPS informazioni sulle sue coordinate geografiche e sul suo orario in ogni condizione meteorologica, ovunque sulla Terra o nelle sue immediate vicinanze . 3, 55

GPX *standard* progettato per il trasferimento di dati GPS tra applicazioni software. Può essere usato per descrivere punti, tracce e percorsi. 47, 55

Hot-reloading funzionalità che permette di aggiornare un applicazione in tempo reale modificando il sorgente, senza dover riavviare il processo in esecuzione. 7

HTTP Protocollo di comunicazione per lo scambio di ipertesti. 38, 55

JSON formato per lo scambio di dati, solitamente utilizzato per le comunicazioni *client-server*. 8, 55

Linter strumento che analizza il codice sorgente per contrassegnare errori di programmazione, bug, errori stilistici e costrutti sospetti. 51

Mockup modello o prototipo dimostrativo incompleto del prodotto finale da realizzare. iii

POI nel contesto applicativo indica un punto di interesse, quindi un luogo rilevante per il *tour*, è definito da una posizione geografica, una descrizione e una tipologia. 2, 55

REST è uno stile architetturale basato su *HTTP* senza ulteriori livelli, per sistemi distribuiti stateless. 32, 55

Server componente software che fornisce servizi e risorse ai *client* che lo richiedono. 6

Sound-null safety proprietà dei linguaggi orientati agli oggetti che garantisce che nessun riferimento a oggetto abbia un puntatore nullo. 7

Sprint intervallo di tempo fisso e ripetibile durante il quale viene creato un prodotto con più alto valore possibile. Di solito, uno Sprint dura un mese o meno. 5

Sprint planning attività che da il via a uno *sprint*, allo scopo di definire gli obiettivi da raggiungere. 5

Sprint review ispezione del lavoro svolto durante lo *sprint*, allo scopo di rilevare possibili errori e determinare le attività successive. 5

SQL linguaggio standardizzato per database basati sul modello relazionale. 8

Tour nel contesto applicativo indica un evento cicloturistico, è formato da un'insieme di giornate, dei partecipanti, un luogo di partenza e un luogo di arrivo. 2

UML in ingegneria del software *UML, Unified Modeling Language* (ing. linguaggio di modellazione unificato) è un linguaggio di modellazione e specifica basato sul paradigma object-oriented. Gran parte della letteratura di settore usa tale linguaggio per descrivere soluzioni analitiche e progettuali in modo sintetico e comprensibile a un vasto pubblico. 11, 55

User story documento che descrive le funzionalità e i requisiti del prodotto dalla parte del cliente. 29

Bibliografia

Siti web consultati

Clustering dei dati con google$_m aps_f lutter$. URL: https://pub.dev/packages/google_maps_cluster_manager (cit. a p. 8).

Dart linter. URL: https://dart.dev/tools/linter-rules (cit. a p. 51).

Documentazione ufficiale di Dart. URL: https://dart.dev/guides.

Documentazione ufficiale di Flutter. URL: https://docs.flutter.dev/ (cit. a p. 9).

Documentazione ufficiale di Sqlite. URL: https://www.sqlite.org/docs.html (cit. a p. 8).

Il formato GPX. URL: https://www.topografix.com/gpx.asp (cit. a p. 48).

Il framework Scrum. URL: https://scrumguides.org/scrum-guide.html (cit. a p. 5).

Il manifesto Agile. URL: http://agilemanifesto.org/iso/it/.

Il pattern BLoC. URL: https://bloclibrary.dev/#/gettingstarted (cit. a p. 9).

Wikipedia. URL: https://www.wikipedia.org/.